

UNIVERSITÀ DEGLI STUDI DI VERONA

CORSO DI LAUREA IN INFORMATICA

Sistemi Operativi

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1	Introduzione ai sistemi operativi	4
1.1	Algoritmi sequenziali e richiami	4
1.2	Organizzazione di un sistema elaborativo	5
1.3	Attività del sistema operativo	6
1.4	Gestione delle risorse	8
2	Strutture dei sistemi operativi	10
2.1	Servizi e interfaccia del sistema operativo	10
2.2	Chiamate di sistema	12
2.3	Servizi di sistema, linker e loader	14
2.4	Struttura del sistema operativo	16
3	Processi	20
3.1	Scheduling e operazioni sui processi	20
3.2	Comunicazione tra processi	26
4	Thread e concorrenza	29
4.1	Programmazione multicore e multithreading	29
4.2	Librerie dei thread	33
4.3	Threading implicito	35
4.4	Problemi di gestione multithread	36
5	Scheduling della CPU	40
5.1	Concetti di base	40
5.2	Algoritmi di scheduling	42
5.3	Scheduling dei thread	45
5.4	Valutazione algoritmi	47
6	Sincronizzazione dei processi	49
6.1	Sezione critica e Peterson	49
6.2	Supporto hardware	52

<i>INDICE</i>	3
6.3 Lock Mutex, Semafori e Monitor	53
6.4 Liveness	58
6.5 Esempi di sincronizzazione	59
7 Stallo dei processi	63
7.1 Stallo e multithread	63
7.2 Caratterizzazioni di stallo	63
7.3 Metodi di gestione e prevenzione	63
7.4 Elusione e rilevamento stallo	63
7.5 Ripristino	63
8 Bash	64
8.1 Programmazione Bash	64
9 API Posix	65
9.1 Gestione dei file	65
9.2 Gestione dei diritti	65
9.3 Lettura/scrittura	65
9.4 Gestione dei processi	65
9.5 Segnali	65
9.6 Comunicazione tra processi tramite PIPE	65
9.7 Pthread	65

Capitolo 1

Introduzione ai sistemi operativi

1.1 Algoritmi sequenziali e richiami

Prima di entrare nel vivo del corso è necessario richiamare alcuni concetti e dinamiche visti nel corso di architetture degli elaboratori. Anzitutto, un **algoritmo sequenziale** è un insieme finito di istruzioni elementari, eseguibili in modo deterministico da una macchina astratta. Tale macchina rappresenta un modello teorico di calcolo e possiede le seguenti caratteristiche:

- **Il calcolo è discreto:** L'esecuzione avviene come sequenza di passi elementari, ciascuno dei quali richiede tempo finito.
- **Il comportamento è deterministico:** A parità di stato iniziale e input, l'evoluzione del calcolo è univocamente determinata.
- **La memoria è idealmente illimitata.**
- **Ogni dato manipolato ha dimensione finita**, ma non esiste un limite prefissato alla quantità totale di memoria utilizzabile.

I suddetti algoritmi sono espressi in un linguaggio di programmazione, al quale è associata una macchina astratta capace di eseguire i programmi scritti in tale linguaggio. In particolare, descriviamo tali macchine tramite tre componenti fondamentali:

- **Memoria:** Insieme idealmente infinito di celle per memorizzazione dati, tale che:
 - Ogni cella è identificata da un indirizzo univoco.
 - L'associazione tra celle e indirizzi è biunivoca.
 - Ogni cella può contenere un valore, ed ogni possibile associazione celle-valori memorizzati fornisce una configurazione particolare della memoria.

- **Ambiente:** Associazione tra un insieme finito di identificatori (variabili) e indirizzi di memoria. Consente di collegare i nomi simbolici utilizzati nel programma alle effettive locazioni di memoria.
- **Interprete:** Meccanismo per l'esecuzione delle istruzioni tramite il ciclo fetch-code-execute.

1.2 Organizzazione di un sistema elaborativo

Il **sistema operativo** è lo strato di software che gestisce le risorse hardware di un elaboratore e fornisce servizi a utenti e programmi applicativi. Svolge il ruolo di intermediario tra hardware e utente, con gli obiettivi principali di rendere conveniente l'uso del sistema per l'utente e garantire un utilizzo efficiente delle risorse hardware. Si tratta di una delle quattro componenti dei sistemi di calcolo moderni, insieme all'hardware, gli applicativi e gli utenti.

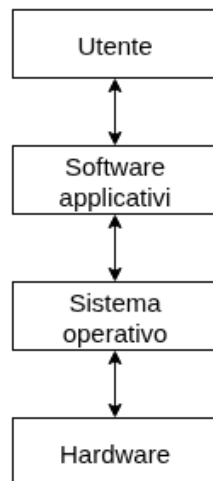


Figura 1.1: Componenti astratte di un elaboratore moderno

La componente centrale del sistema operativo è il **kernel**, un processo eseguito in modalità privilegiata costantemente in background per fornire meccanismi fondamentali per la gestione degli altri processi e delle risorse. Gestisce direttamente, tra l'altro, CPU, memoria e periferiche I/O.

I programmi che forniscono funzionalità di base non facenti parte del kernel sono detti **programmi di sistema** e sono per esempio la shell o i compilatori, mentre gli altri programmi sono detti **applicativi**. Molti sistemi moderni includono inoltre uno strato software aggiuntivo detto **middleware**, che fornisce servizi di alto livello come database, sistemi grafici o infrastrutture distribuite. All'accensione del calcolatore, l'esecuzione

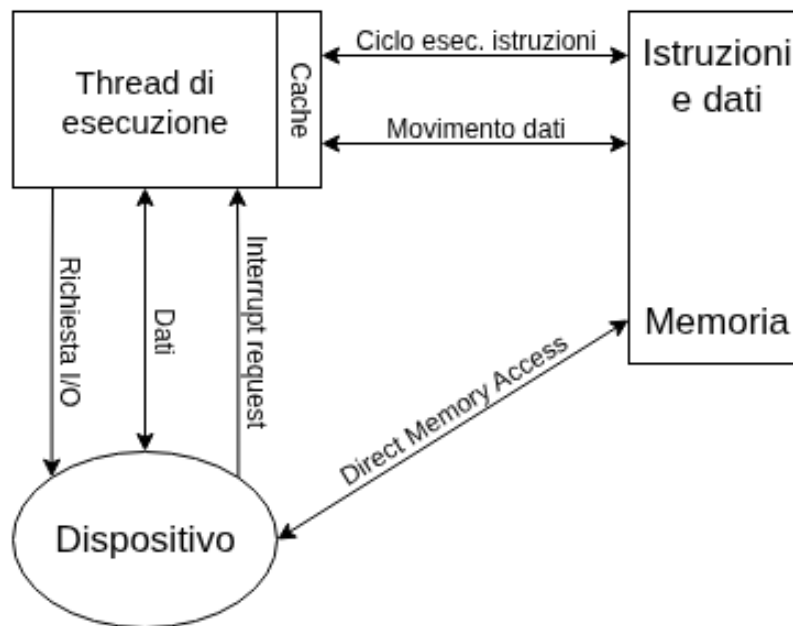


Figura 1.2: Funzionamento di un elaboratore moderno

inizia da un programma **bootloader** residente nel firmware (BIOS o UEFI), il cui compito è inizializzare l'hardware e caricare in memoria un programma di **bootstrap**. Il bootloader provvede quindi a caricare il kernel del sistema operativo in memoria e a trasferirgli il controllo.

Inizializzato, il kernel configura le strutture dati fondamentali, inizializza i dispositivi e avvia i primi processi di sistema. Terminata questa fase, il sistema entra in una modalità di esecuzione guidata dagli eventi. Il controllo del processore passa tra processi e kernel in risposta a interrupt hardware, eccezioni oppure trap, quindi richieste di servizio da parte dei programmi utente.

1.3 Attività del sistema operativo

Fra le principali caratteristiche dei sistemi operativi moderni vi è inoltre la **multiprogrammazione**. Un singolo programma non utilizza costantemente la CPU: durante le operazioni di input/output o in attesa di eventi, il processore rimarrebbe inattivo. Con questo meccanismo si aumenta il suo utilizzo mantenendo in memoria più programmi contemporaneamente.

Il sistema operativo salva temporaneamente in memoria centrale alcuni programmi, in questo caso detti **processi**, e ne sceglie uno per l'esecuzione. Quando il processo scelto deve attendere un evento, la CPU viene assegnata ad un altro processo pronto. Dandole

costantemente nuovi compiti quando sono disponibili si riduce il sul tempo di inattività.

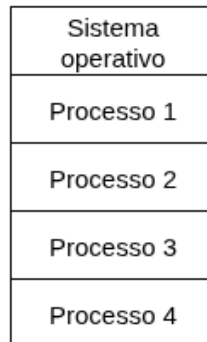


Figura 1.3: Configurazione di memoria per sistemi multiprogrammati

Un'estensione logica della multiprogrammazione è il **multitasking**, detto anche time sharing. Si tratta di un modello nel quale la CPU è assegnata ai processi per intervalli di tempo molto piccoli e limitati, detti **quanti**. Il cambio tra i processi è gestito dallo **scheduler** del processore, ed è effettuato ad una frequenza tale da creare l'impressione di un'esecuzione simultanea, garantendo buoni tempi di risposta.

Poiché le risorse hardware sono condivise, esistono alcuni meccanismi per garantire un'efficiente esecuzione dei programmi, come anche la protezione del sistema operativo. Il primo è dato dalla **memoria virtuale**, la quale consente di eseguire processi non completamente salvati in memoria fisica tramite una sua astrazione. Il secondo riguarda la **modalità duale di funzionamento**, che garantisce protezione tra sistema operativo e programmi utente; in particolare distingue tramite un **bit di modalità** le:

- **Modalità kernel 0**: Per eseguire istruzioni privilegiate.
- **Modalità utente 1**: Per eseguire istruzioni accessibili all'utente.

Siccome l'accesso alle istruzioni critiche è precluso agli utenti, quando un programma necessita di un servizio del sistema operativo, invoca una **system call**. Questa genera una **trap**, vista dal sistema come interruzione, e l'hardware passerà in modalità kernel. Il controllo passa ora a una routine del sistema operativo, individuata tramite un vettore con i valori relativi alle trap. Si esegue ora il servizio richiesto e il controllo ritorna al programma utente in user mode.

Se durante l'esecuzione si verifica una condizione anomala, come per esempio un accesso a memoria non valida o divisione per zero, l'hardware genera un'**eccezione**. In tal caso il controllo passa al sistema operativo, che può terminare il processo, segnalare l'errore ed eseguire eventuali azioni correttive. In alcuni sistemi, se la funzione è presente, possono essere generati dei file testuali detti **memory dump** per analisi successive.

Un ulteriore meccanismo di protezione è rappresentato dal **timer** hardware; un contatore che al raggiungimento dello zero genera un interrupt. Si tratta di una componente essenziale per in primo luogo implementare il time sharing, ma anche per impedire che un processo monopolizzi la CPU, garantendo che il controllo ritorni al sistema operativo entro un certo intervallo di tempo.

1.4 Gestione delle risorse

Per svolgere i propri compiti, un processo richiede alcune risorse, come memoria, file o tempo di CPU. Queste possono essergli assegnate alla sua nascita oppure in esecuzione, per poi essere rese al sistema operativo al termine del processo. In particolare diciamo che i programmi sono entità passive, mentre i processi sono attivi; questi ultimi si dividono in:

- **Single-threaded:** Contengono un solo thread di esecuzione, quindi un unico program counter.
- **Multi-threaded:** Contengono più fili di esecuzione, ciascuno con il proprio program counter e stack.

Tipicamente i sistemi attuano più processi concorrentemente; ciò ha luogo grazie ad un multiplexing della CPU tra **processi e thread**. Per quanto riguarda la loro **gestione**, i sistemi operativi sono responsabili della creazione e cancellazione dei processi utenti e di sistema, schedulazione di processi e thread sulle CPU, sospensione e ripristino dei processi ed infine della fornitura di meccanismi per la loro sincronizzazione e comunicazione.

Per eseguire un programma è necessario che almeno una parte delle sue istruzioni e relativi dati sia scritta nella memoria, in quanto è necessario accedervi per il loro funzionamento. Il programma genera così gli indirizzi logici, che verranno tradotti in indirizzi fisici dal sistema di **gestione della memoria**. In tal merito, il sistema operativo è responsabile di:

- Tenere traccia di quali zone di memoria sono usate e da chi.
- Assegnare e revocare lo spazio di memoria in base alle necessità.
- Decidere quali processi e dati debbano essere caricati in memoria centrale o trasferiti in quella di massa.

Il sistema operativo fornisce inoltre un'interfaccia logica uniforme per la memorizzazione delle informazioni, detta **file**. Si tratta di un'astrazione logica per la memorizzazione indipendente dal dispositivo fisico. Ogni file è organizzato in **directory**, permettendo di ottenere una struttura gerarchica, e possono prendere più forme: **libera**, come per i file testuali, oppure **strutturata**, come per gli mp3. Per quanto riguarda la **gestione dei file**, il sistema operativo è responsabile di:

- Creazione e cancellazione di file e directory.
- Fornitura delle funzioni fondamentali per la gestione di file e directory.
- Associazione dei file ai dispositivi di memoria secondaria.
- Creazione di backup dei file su dispositivi di memorizzazione non volatili.

Poiché i programmi utilizzano i dispositivi di memoria secondaria come dischi rigidi per memorizzare dati e programmi, è di fondamentale importanza che sia gestita correttamente e in modo efficiente. Per la **memoria di massa**, infatti, il sistema operativo **gestisce**:

- Montare e smontare (mounting, unmounting) le unità di memoria.
- Assegnazione dello spazio e gestione dello spazio libero.
- Scheduling del disco.
- Partizionamento.
- Protezione.

Come già detto, più utenti possono usare uno stesso calcolatore ed è quindi necessario gestire l'accesso ai dati mediante apposite regole. Ciò avviene con la **protezione**: meccanismi di controllo dell'accesso alle risorse del PC da parte di processi o utenti. Ci sono varie strategie per attuarla, ma tutte devono fornire le specifiche dei controlli e gli strumenti utilizzati. Un altro concetto molto importante, a causa del costante incremento di attacchi informatici, è la **sicurezza**: la difesa del sistema da attacchi volti al sistema operativo.

Questi due concetti presuppongono che si riesca a distinguere i propri utenti da quelli non autorizzati; generalmente ciò avviene grazie agli identificatori utente, che garantiscono loro unicità. Quando un utente o un gruppo si collega al sistema, infatti, è presente una fase di autenticazione per determinare l'ID corretto e associarlo al processo dell'utente. Se richiesto, è inoltre possibile cambiare l'ID effettivo tramite un comando, ottenendo accesso a comandi privilegiati.

Come ultimo concetto presentiamo la **virtualizzazione**, una tecnica che permette di astrarre l'hardware di un singolo computer, quindi tutte le sue componenti, in diversi ambienti di esecuzione, creando l'illusione di avere più sistemi operativi attivi allo stesso momento su un solo calcolatore, i quali sono capaci di interagire l'uno con l'altro. Otteniamo questa dinamica con le **macchine virtuali**, che consentono agli utenti di cambiare agevolmente il sistema operativo del quale si ha necessità. Un concetto simile è quello dell'**emulazione**, ovvero la simulazione tramite software di un hardware diverso da quello del calcolatore; tecnica usata spesso quando il tipo di CPU della macchina è diverso da quello desiderato.

Capitolo 2

Strutture dei sistemi operativi

2.1 Servizi e interfaccia del sistema operativo

I sistemi operativi garantiscono un ambiente in cui eseguire i programmi e fornire servizi, con lo scopo di facilitare l'interazione con i programmatori di applicazioni. Un primo insieme di servizi riguarda funzionalità utili per gli utenti:

- **Interfaccia con l'utente:** Meccanismo di interazione fra utente e sistema, si divide in **interfaccia grafica utente** GUI, **interfaccia a linea di comando** CLI, e per i dispositivi mobile, **interfaccia touch-screen**.
- **Esecuzione programmi:** La possibilità di caricare un programma in memoria, eseguirlo e terminarlo.
- **Operazioni I/O:** Il sistema operativo fornisce un'interfaccia uniforme per operazioni di input/output su dispositivi quali dischi, tastiera, schermo e rete.
- **Gestione file system:** Lettura, scrittura e ricerca file, creazione ed eliminazione di directory. Fornisce inoltre i metadati.
- **Comunicazioni:** Scambiare informazioni in uno stesso calcolatore e in diversi elaboratori connessi da una rete.
- **Rilevamento errori:** Rilevazione e segnalazione di errori riguardanti la CPU, dispositivi di memoria, I/O o programmi utente. Gestiti mediante arresto del programma o del sistema, oppure con un codice errore.

Un secondo gruppo invece è atto all'efficienza del sistema:

- **Allocazione risorse:** Se più utenti o più processi sono attivi in contemporanea, il sistema operativo provvede alla distribuzione delle risorse necessarie a ciascuno di essi.

- **Accounting:** Mantenere traccia dei programmi usati e quante risorse impiegano.
- **Protezione e sicurezza:** Il sistema operativo evita che i processi interferiscano fra di loro e il sistema stesso e verifica gli utenti tramite richieste di autenticazione.

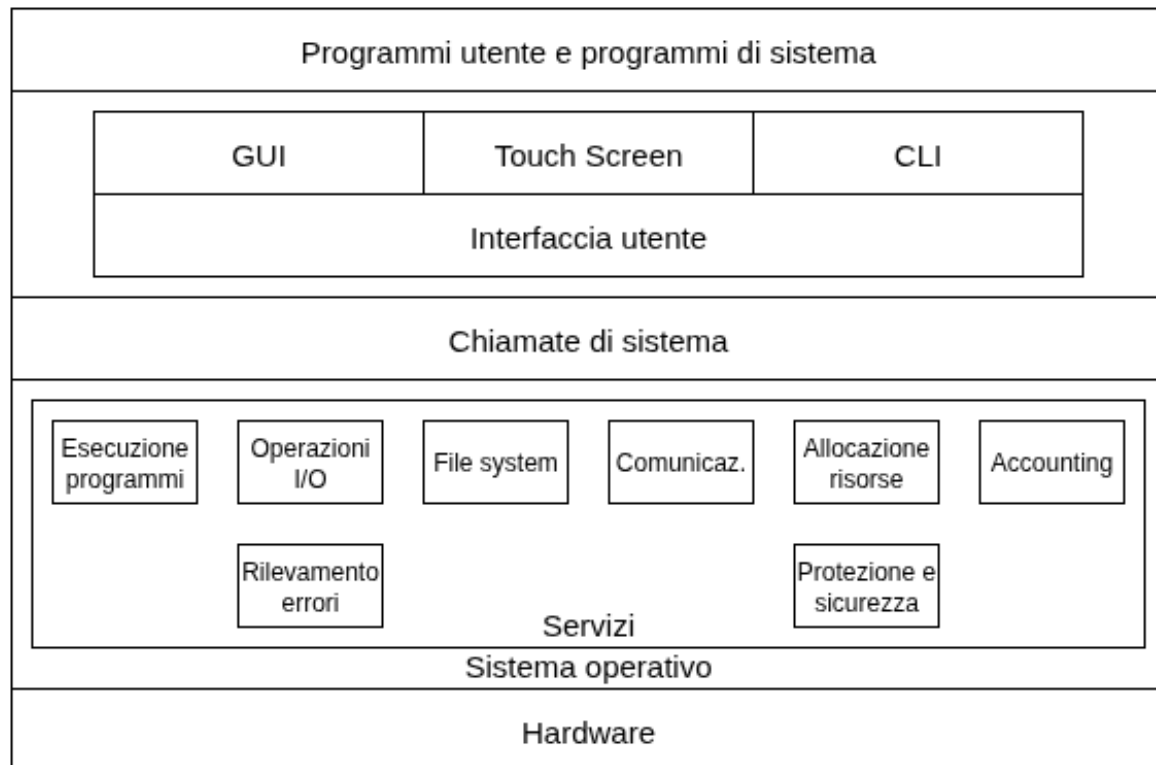


Figura 2.1: Panoramica dei servizi del sistema operativo

Esistono poi due modi fondamentali per gli utenti di comunicare con il sistema operativo: tramite l'**interprete dei comandi** (CLI), implementata da un programma detto **shell**, che permette agli utenti di inserire direttamente le istruzioni da eseguire, e le **interfacce grafiche con l'utente** (GUI), che serve da tramite tra utente e sistema.

Nella maggior parte dei sistemi operativi l'interprete dei comandi è un programma speciale che si attiva all'avvio di un job o al log-in di un utente. La sua funzione principale è raccogliere ed eseguire il comando impartito dall'utente, implementandoli in due modi possibili:

- Se l'interprete contiene il codice per l'esecuzione del comando, come nei casi di gestione file system, salta alla sezione apposita ed invoca le opportune chiamate senza ricorrere a programmi esterni.

- Se non contiene il codice, l'interprete implementa il comando per mezzo di programmi di sistema; non conosce il significato del comando ma si limita ad impiegare il nome per identificare un file da caricare in memoria ed eseguirlo.

Il metodo più utilizzato è il secondo, perché consente la scrittura di nuovi comandi per l'interprete creando nuovi file con un nome appropriato, senza modificare il suo codice.

Per quanto riguarda le interfacce grafiche con l'utente, abbiamo un ambiente nettamente più user-friendly, costituito da una o più finestre con i relativi menu entro cui muoversi con il puntatore del mouse. Usa la metafora della scrivania, il **desktop**, per rendere intuitivi i concetti. Qui si possono trovare le varie **icone**, le quali rappresentano programmi, file, directory (chiamate **cartelle**) e funzioni del sistema. Si tratta del metodo di interazione con l'utente più utilizzato in generale, sebbene i moderni sistemi operativi diano l'opzione per utilizzare entrambi.

2.2 Chiamate di sistema

Le **chiamate di sistema** (system calls) costituiscono un'interfaccia per i servizi resi disponibili dal sistema operativo. Le applicazioni vi possono accedere tramite **Application Programming Interface** (API), fornite dalle librerie digitali, e specificano un insieme di funzioni capaci di comunicare con il sistema operativo. Le API più diffuse sono quella di Windows, POSIX, standard usato per i sistemi UNIX, e quella di Java.

Un altro fattore importante è dato dall'**ambiente di esecuzione al run-time**; una suite di programmi che consente l'esecuzione di applicativi scritti in un determinato linguaggio, collegandolo alle chiamate di sistema tramite un'apposita interfaccia. In base al linguaggio utilizzato, si compone di compilatori, interpreti, librerie digitali, linker e loader.

Ogni chiamata è codificata da un numero specifico, salvato nella **system call table**, a sua volta mantenuta nel kernel e usata dall'**interfaccia alle chiamate di sistema**. Quest'ultima invoca la syscall richiesta, restituendone lo stato. Si possono inoltre presentare in modi diversi, e dipendentemente dal sistema operativo può essere necessario fornire più informazioni come parametri, con uno dei seguenti modi:

- Passaggio del singolo parametro a registro.
- Passaggio di un indirizzo di blocco di memoria a registro.
- Push sulla stack dei parametri.

Inoltre, possiamo raggruppare le chiamate a sistema in sei categorie principali: controllo processi, gestione file, gestione dispositivi, gestione informazioni, comunicazione e protezione. Analizziamole nel dettaglio, fornendo nomi illustrativi delle varie chiamate:

- **Controllo dei processi:** Un programma deve potersi fermare in modo normale o anomalo (*end()*, *abort()*). In quest'ultimo caso, spesso viene generato un **memory dump**, analizzabile con un debugger. Un processo che esegue un programma può inoltre richiedere di caricare o eseguire (*load()*, *execute()*) altri programmi tramite una specifica richiesta data da comando utente, clic del mouse o un comando batch. Qui possiamo avere due casi di studio:

Se al termine del nuovo programma il **controllo torna nel chiamante**, è necessario salvare una sua immagine della memoria; mentre se invece i programmi continuano l'**esecuzione in contemporanea** diciamo che si è creato un nuovo processo in multiprogrammazione *create_process()*.

Naturalmente è necessario poterne mantenere il controllo e quindi determinare o reimpostare i vari attributi (*get_process_attributes()*, *set_process_attributes()*), come anche terminare il processo (*terminate_process()*). La terminazione può dipendere dal verificarsi di un certo evento (*wait_event()*) oppure dopo aver aspettato un determinato tempo (*wait_time()*), ma in entrambi i casi bisogna segnalare l'evento (*signal_event()*). Capita spesso inoltre che i processi possano condividere dati, rendendo necessario assicurarne l'integrità ed evitare che vengano acceduti o elaborati da più processi allo stesso tempo (*acquire_lock()*, *release_lock()*).

- **Gestione dei file:** È necessario poter creare e cancellare file (*create()*, *delete()*), come anche directory. Su questi è possibile eseguire le operazioni di apertura (*open()*), lettura (*read()*), scrittura (*write()*), riposizionamento (*reposition()*) e chiusura (*close()*). Si richiede inoltre di poter determinare i valori dei loro attributi, come nome, tipo o codici di protezione (*get_file_attributes()*, *set_file_attributes()*). Ciò si ottiene utilizzando delle API apposite.
- **Gestione dei dispositivi:** Per essere eseguito, un processo richiede alcune risorse come spazio in memoria, su disco o accesso a file. Se disponibili, si possono concedere ed il controllo potrà ritornare al processo utente, altrimenti deve attendere fin quando non se ne avranno a sufficienza.

In caso di utenti multipli, il sistema operativo potrebbe fare una richiesta per l'uso esclusivo di un dispositivo (*request()*), per poi rilasciarlo una volta terminato l'utilizzo (*release()*). Se in ambiente UNIX, è possibile invocare le stesse chiamate a sistema utilizzate per i file, in quanto è uno dei sistemi operativi che li combina in un'unica macrostruttura. Si tratta di una scelta progettuale e non universale.

- **Gestione delle informazioni:** Queste sono chiamate di sistema il cui scopo è trasferire informazioni tra programmi utente e sistema operativo. Alcuni esempi sono l'ottenimento di data e ora oppure la richiesta di effettuare un memory dump (*dump()*). Il sistema operativo contiene informazioni su tutti i suoi processi e sono ottenibili con le chiamate (*get_process_attributes()*, *set_process_attributes()*).

- **Comunicazione:** Per quanto riguarda lo scambio di informazioni esistono due modelli principali:

Il **modello a scambio di messaggi** vede processi comunicanti che condividono informazioni tramite una **mailbox** comune. Per prima cosa, si apre un collegamento fra due entità comunicanti il cui nome deve essere noto ed identificato con il **nome di macchina** (*get_hostid()*) e **nome di processo** (*get_processid()*). Questi due identificatori sono poi passati alle chiamate a sistema per aprire e chiudere il collegamento, se è accettato dall'altra entità (*accept_connection()*). Spesso i processi che gestiscono la comunicazione sono demoni specifici; in particolare l'origine dell'informazione è detta **client** ed il ricevente è il **server**. Si servono delle chiamate *read_message()* e *write_message()* per lo scambio e *close_connection()* per terminare la comunicazione.

Nel **modello a memoria condivisa** invece, si crea un'area di memoria condivisa da più processi (*shared_memory_create()*, *shared_memory_attach()*). Questo spazio non è gestito dal sistema operativo ed è necessario che i processi si accordino per non andare in conflitto. Risulta utile perché uno scambio di informazioni su uno stesso elaboratore permette di lavorare alla velocità della memoria e rende il processo più semplice, ma presenta problemi riguardo la protezione e la sincronizzazione dei processi.

- **Protezione:** Il meccanismo di controllo per l'accesso alle risorse del calcolatore. Consente di modificare i permessi di accesso (*set_permission()*, *get_permission()*) a risorse come file e dischi, come anche gestire quelli di utenti specifici, per limitare accessi non autorizzati (*allow_user()*, *deny_user()*).

2.3 Servizi di sistema, linker e loader

I **servizi di sistema** (o system utilities) sono programmi forniti con il sistema operativo che offrono un ambiente per lo sviluppo, l'esecuzione e la gestione dei programmi. In particolare, si classificano nelle seguenti categorie:

- **Gestione dei file:** Compiono operazioni su file e directory come la loro creazione, cancellazione, copia, rinomina e spostamento.
- **Informazioni di stato:** Programmi che forniscono informazioni sullo stato del sistema, come processi attivi, utilizzo della memoria, data e ora o statistiche di prestazione.
- **Modifica dei file:** Programmi per la modifica del contenuto dei file; gli editor di testo.

- **Ambienti di supporto per la programmazione:** Compilatori, assembleri o interpreti. Spesso forniti direttamente con il sistema operativo oppure installabili separatamente.
- **Caricamento ed esecuzione dei programmi:** Una volta assemblato o compilato il programma, deve essere caricato in memoria per l'esecuzione. Questi programmi sono detti **loader**.
- **Comunicazioni:** Programmi che forniscono meccanismi per creare collegamenti virtuali tra processi, utenti e calcolatori diversi. Consentono lo scambio di informazioni tra processi, utenti o calcolatori attraverso la rete.
- **Servizi in background:** I processi di sistema costantemente in esecuzione, i quali terminano il lavoro all'arresto del sistema. Un esempio è il demone per la gestione delle interfacce di rete.

Oltre a questi servizi, molti sistemi operativi includono ulteriori programmi di utilità per attività comuni, come editor di testo, strumenti di archiviazione e browser web. Generalmente i programmi risiedono su disco in forma di file binario o eseguibile e per essere eseguito dalla CPU è necessario che venga caricato in memoria e associato a un processo. Non è un processo istantaneo, quindi analizziamo fase per fase.

Anzitutto, i **file sorgenti** vengono compilati in **file oggetto rilocabili**, progettati per essere caricati in una qualsiasi posizione della memoria fisica. Il **linker** combinerà ogni file oggetto ed eventuali librerie necessarie in un singolo **binario** eseguibile. Questo è caricato in memoria tramite un **loader** e, allo stesso tempo, avviene la **rilocalizzazione**, che assegna gli indirizzi definitivi alle componenti del programma e riaggiusta il codice e i dati nel programma in base a essi, per garantirne un corretto funzionamento.

Prendiamo ora come esempio l'ambiente UNIX. Per eseguire un programma è necessario dare il relativo comando alla shell. Qui il sistema operativo crea un nuovo processo con la chiamata *fork()* e richiama il loader con *exec()*, passando il nome del file eseguibile. Viene quindi caricato il programma in memoria usando lo spazio di indirizzamento del processo appena creato. File oggetto ed eseguibili seguono un formato standard che dipende dal sistema operativo di riferimento:

- **UNIX:** Formato Executable and Linkable Format (ELF)
- **Windows:** Portable Executable (PE)
- **MacOS:** Mach-O

Bisogna ricordare che le applicazioni compilate su un sistema operativo non sono generalmente compatibili con altri. Infatti, ogni sistema fornisce un insieme univoco di

chiamate a sistema e una specifica **interfaccia binaria** (ABI), il quale è diversa in altri ambienti. Nonostante ciò, esistono tre modi per rendere disponibili le applicazioni su più sistemi:

- **Uso di un linguaggio interpretato:** I linguaggi come Python o Ruby funzionano grazie ad un interprete; questo legge le righe del codice ed esegue istruzioni equivalenti all'insieme di istruzioni nativo, invocando le chiamate del sistema operativo di riferimento. Dato questo passaggio di traduzione, le prestazioni risultano inferiori rispetto ad applicazioni compilate.
- **Uso di un linguaggio che si serve di macchina virtuale:** Linguaggi come Java utilizzano una macchina virtuale per fornire un ambiente di esecuzione applicazioni indipendentemente dal sistema operativo.
- **Uso di un linguaggio o API standard:** Scrivendo applicazioni che utilizzano interfacce standard come POSIX, è possibile ricompilarle su sistemi diversi con modifiche minime. Sebbene ciò garantisca prestazioni ottimali, il porting può richiedere molto tempo poiché è necessaria una ricompilazione e un adattamento al sistema di destinazione per ogni nuova versione del software.

2.4 Struttura del sistema operativo

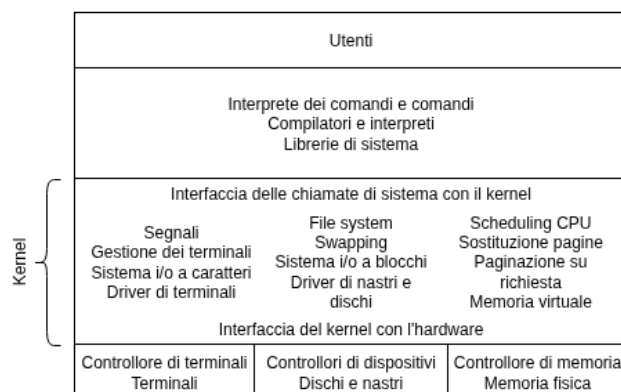
La realizzazione di un sistema operativo vede storicamente protagonista il linguaggio Assembly, ma ad oggi ne vengono normalmente usati di più ad alto livello, come C e C++. Ciò comporta dei vantaggi diretti, come l'avere un codice più compatto, leggibile e più facilmente adattabile ad altre architetture, ma anche svantaggi, come una minore velocità di esecuzione e maggiore utilizzo di spazio di memoria. Tuttavia, grazie alle prestazioni dell'hardware moderno e all'utilizzo di algoritmi e strutture dati migliori, si tratta di problemi ignorabili.

In ogni caso, è necessario prima verificare il corretto funzionamento del sistema operativo, per poi procedere con il miglioramento delle prestazioni, individuando i colli di bottiglia che aumentano i tempi di esecuzione, normalmente le procedure come il gestore degli interrupt, dell'I/O, della memoria e lo scheduler della CPU.

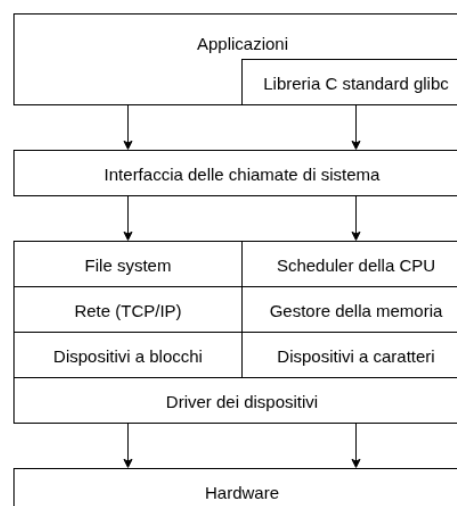
Affinché un sistema operativo possa funzionare ed essere facilmente modificato, sono stati ideati diversi modelli per definirne delle strutture ben precise. Un primo esempio è dato dalla **struttura monolitica**: l'assenza di struttura. Vede tutte le funzionalità del sistema implementate all'interno del kernel, le quali operano nello stesso spazio di indirizzamento.

I vantaggi di questo approccio risiedono nella velocità di comunicazione all'interno del kernel e di un overhead ridotto dell'interfaccia alle chiamate di sistema. Si tratta di una

struttura utilizzata dallo UNIX originale e da Linux, il quale vedremo presenta anche una struttura **modulare**, quindi con delle sottocomponenti che definiscono le parti del sistema.



(a) Struttura di UNIX



(b) Struttura di Linux

La struttura monolitica è caratterizzata da componenti **fortemente accoppiate**, perché tutto è descritto in un solo insieme. Tuttavia è possibile ideare una struttura dai moduli separati, quindi **debolmente accoppiata**, il cui loro insieme compone il kernel. Il vantaggio principale risiede nel mantenimento: le modifiche su una componente non influiscono sulle altre, rendendo notevolmente più semplice il debugging e consentendo una maggiore libertà nella creazione e modifica del codice.

Un'idea che applica questo concetto è l'**approccio stratificato**, il quale vede il sistema operativo diviso in livelli, dove quello più basso rappresenta l'hardware e quello più alto l'interfaccia con l'utente. Ogni strato è composto da strutture dati e un insieme di routine richiamabili dallo strato superiore. La progettazione stessa del sistema operativo sarà guidata da questa struttura; infatti, si passa agli strati superiori solamente quando tutte le funzionalità dei livelli inferiori funzionano correttamente. Sebbene sia decisamente una struttura sicura, in termini di prestazioni risulta scarsa a causa dell'overhead dovuto dall'attraversamento di più strati e per questa ragione non è l'unica idea fondante i sistemi operativi odierni. Tuttavia, si tende ad applicare una parziale stratificazione dove è presente un numero minore di livelli, ognuno con più funzioni. Ciò consente di evitare i problemi connessi alla definizione dell'interazione fra gli strati.

Un altro approccio è quello orientato a **microkernel**: tutti i componenti non essenziali vengono rimossi dal kernel e sono realizzati come programmi di livello utente e di sistema. Ciò che rimane nel kernel di dimensioni ridotte sono i servizi minimi di comunicazione, scheduling e gestione della memoria. Lo scopo principale è quello di fornire

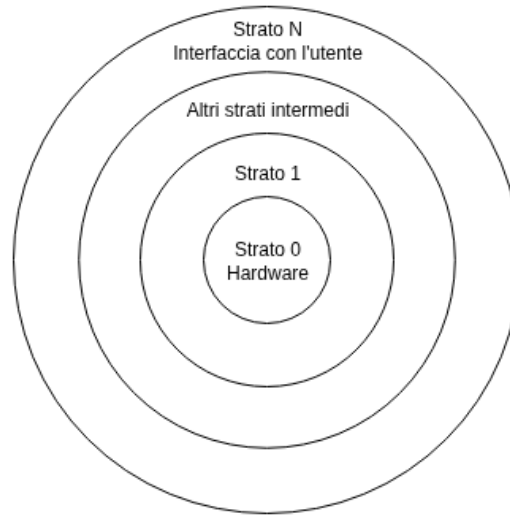


Figura 2.3: Approccio a strati

funzioni di comunicazione tramite scambio di messaggi. Infatti, tra il client e il server, il kernel prende il ruolo dell'intermediario che permette ai due di comunicare.

I principali vantaggi risiedono nella facilità di estensione del sistema operativo e le modifiche minime al kernel quando deve essere aggiornato o modificato. Tuttavia è possibile che presenti cali di prestazioni a causa dell'aumento dell'overhead. Esempi pratici di questo approccio sono sistemi come macOS basati su **Darwin**.

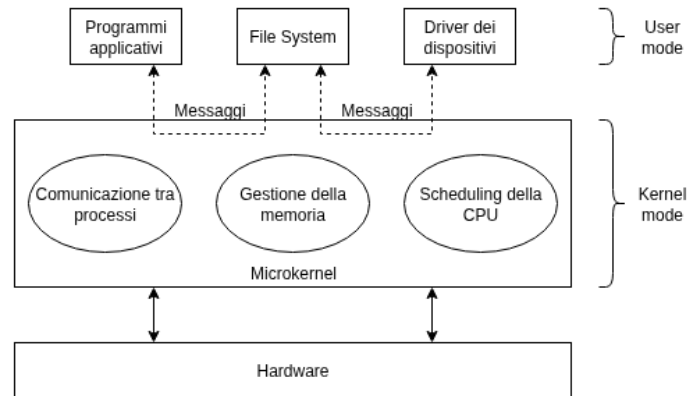


Figura 2.4: Architettura a microkernel

L'approccio attualmente più utilizzato è sicuramente quello basato sui **moduli** del kernel caricati dinamicamente. Il kernel fornisce insiemi di componenti di base integrati da funzionalità aggiunte dinamicamente quando è in esecuzione. Ricorda un sistema stratificato più flessibile con idee di microkernel, perché ogni modulo può chiamare qualsiasi

altro e il kernel è ridotto alle funzioni minime. Si tratta di un approccio usato per le implementazioni moderne di Linux, Windows e macOS. Per esempio, Linux utilizza **moduli del kernel caricabili** (LKM), i quali sono aggiunti quando necessario per supportare i driver dei dispositivi e i file system. Ciò consente di avere un kernel modulare e dinamico, mantenendo allo stesso tempo le prestazioni di un sistema monolitico.

I sistemi operativi odierni non adattano una sola idea architetturale, bensì ne applicano più alla volta. Per esempio, come menzionato prima, Linux utilizza di base un approccio monolitico, ma è allo stesso tempo modulare poiché consente di caricare componenti al kernel dinamicamente. Anche Windows presenta una struttura simile, applicando tuttavia idee orientate ai microkernel, come per esempio il supporto a sottosistemi separati.

Capitolo 3

Processi

3.1 Scheduling e operazioni sui processi

Si ritiene necessario dare un nome alle attività compiute dalla CPU: in particolare, i sistemi **batch** eseguono **job**, mentre quelli **time-sharing** dei **task**. Chiamiamo queste attività col nome generale di **processo**.

Intuitivamente, un processo è un programma in esecuzione il cui stato è rappresentato da program counter e registri del processore. Si compone di:

- **Sezione di testo:** Contiene il codice eseguibile.
- **Sezione dei dati:** Contiene le variabili globali.
- **Heap:** Memoria allocata dinamicamente durante l'esecuzione del programma.
- **Stack:** Memoria usata temporaneamente nelle chiamate a funzione.

È importante notare che heap e stack sono di dimensione variabile e crescono l'una verso l'altra; è compito del sistema operativo garantire che non si sovrappongano. La prima aumenta o diminuisce di dimensione grazie a funzioni come malloc e free per il linguaggio C, mentre le chiamate a funzione inseriscono un **record** di attivazione contenente parametri, variabili locali e indirizzo di ritorno sulla stack, per poi rimuoverlo quando il controllo del programma ritorna al chiamante.

Noi sappiamo che un programma diventa processo quando il suo eseguibile è caricato in memoria e sebbene due processi possono essere associati ad uno stesso programma, sono da considerare come due istanze diverse, le quali seguono il proprio percorso di esecuzione, detto **thread**. Inoltre, durante la loro vita, i processi possono presentarsi nei seguenti stati:

- **Nuovo:** Il processo è appena creato.

- **In esecuzione:** Le istruzioni del processo stanno venendo eseguite.
- **In attesa:** Il processo attende il verificarsi di un evento.
- **Pronto:** Il processo attende di essere assegnato a un'unità di elaborazione.
- **Terminato:** Il processo ha terminato l'esecuzione.

Questi stati sono comuni a tutti i sistemi operativi, sebbene alcuni introducano ulteriori distinzioni. È bene inoltre notare che sebbene più processi possano essere pronti, i core delle unità di elaborazione possono eseguirne solo uno per volta.

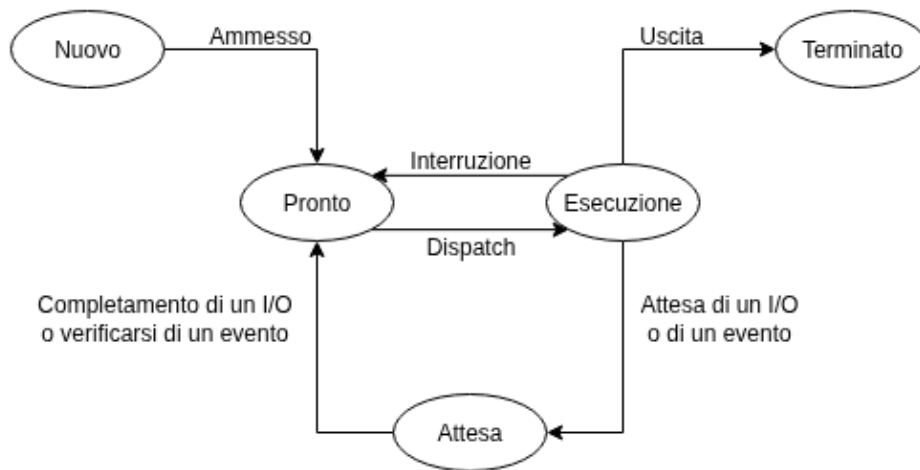


Figura 3.1: Diagramma degli stati di un processo

Nei sistemi operativi i processi si rappresentano con **blocchi di controllo** (Process Control Block, PCB), depositi di informazioni per ogni processo specifico. Salvano dati in merito allo stato corrente, il valore del program counter, lo stato dei registri della CPU e informazioni sul suo scheduling, informazioni sulla gestione della memoria, di accounting e infine sullo stato dell'I/O.

Nei sistemi operativi moderni si è esteso il concetto di processo introducendo la possibilità di avere più percorsi di esecuzione, permettendo ai processi di svolgere più compiti alla volta. In un sistema che supporta i thread, infatti, il PCB è esteso per includere informazioni su di essi.

Riprendiamo ora il concetto di **multiprogrammazione**, il cui obiettivo è avere almeno un processo sempre in esecuzione per massimizzare l'utilizzo della CPU. Il cambio tra i vari processi avviene così frequentemente da consentire l'interazione con ciascun programma durante la loro esecuzione ed è gestita dallo **scheduler**, il quale seleziona il

processo e lo assegna al core della CPU. Le architetture odierne comprendono CPU a più core, ed in particolare chiamiamo **grado di multiprogrammazione** il numero di processi in memoria in un dato istante.

Il bilanciamento tra multiprogrammazione e time sharing richiede di tener conto anche del comportamento complessivo dei processi: definiamo infatti **I/O-bound** i processi che impiegano la maggior parte del tempo nell'esecuzione di operazioni input/output e **CPU-bound** quelli che eseguono nella maggior parte elaborazioni. Ogni processo è inizialmente inserito dallo scheduler in una **coda dei processi pronti**, tipicamente implementata come una struttura dati, in attesa di essere assegnato ad un core. Esiste inoltre una **coda di attesa** dove sono spostati i processi in attesa del completamento di un evento, come la ricezione di un input.

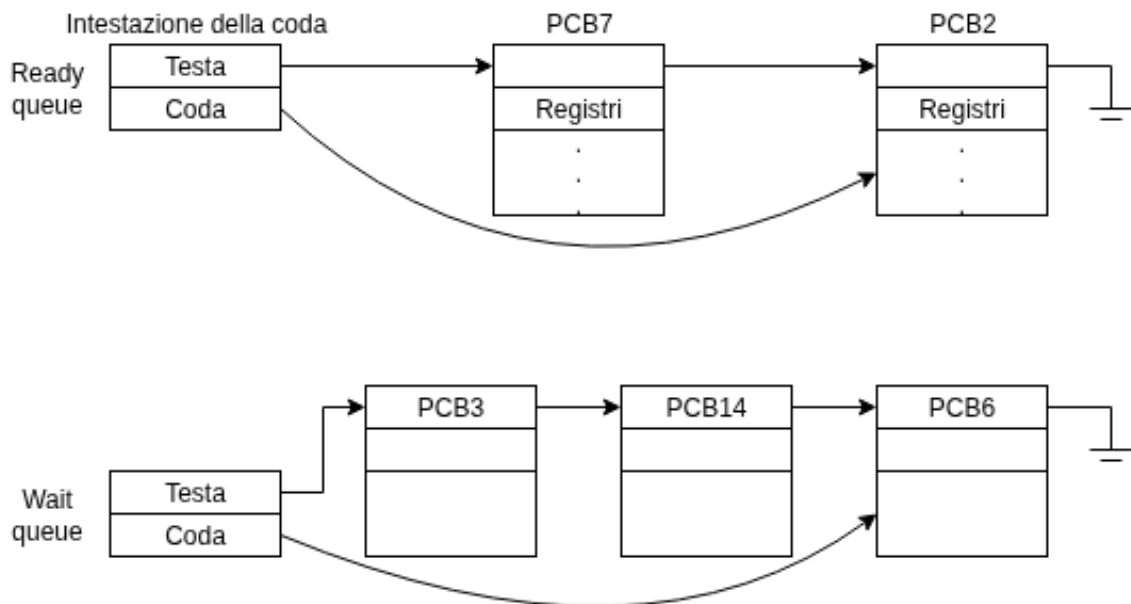


Figura 3.2: Coda dei processi pronti e di attesa

Con l'inserimento nella coda dei processi pronti, si possono verificare i seguenti casi:

- Processo emette richiesta I/O, viene spostato quindi nella coda di I/O.
- Processo crea un processo figlio, attendendo la sua terminazione.
- Processo rimosso forzatamente dalla CPU a causa di interruzione. Viene reinserito nella coda dei processi pronti.

Nei primi due casi, al completamento della richiesta I/O o al termine del processo figlio, il processo originale passa dallo stato di attesa a quello pronto ed è nuovamente inserito

nella coda dei processi pronti. Il ciclo continua fino al termine dell'esecuzione e si rimuoverà quindi da tutte le code, deallocando anche il PCB e le risorse legate al processo.

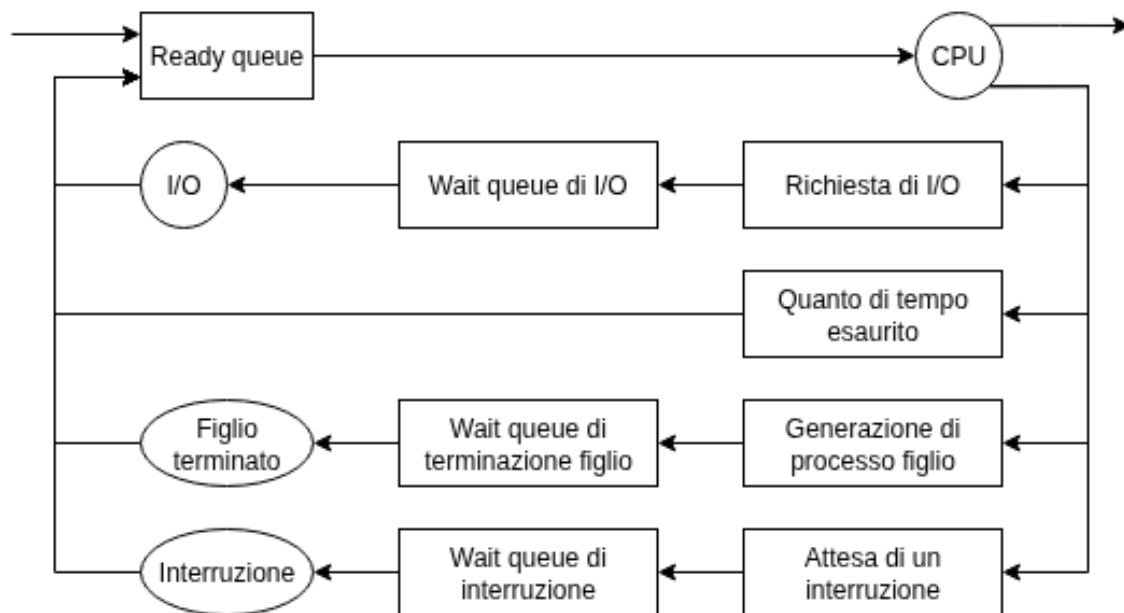


Figura 3.3: Diagramma di accodamento per lo scheduling

Come già detto, i processi si spostano tra le varie code disponibili frequentemente; alcuni sistemi operativi, per ridurre il grado di multiprogrammazione, implementano un algoritmo intermedio nello scheduling per rimuovere forzatamente i processi dalla CPU, detto **avvicendamento dei processi in memoria**.

Viene quindi salvato lo stato al momento dello stop del processo rimosso, per poi riprendere da dove è stato interrotto quando lo si reinserisce in memoria. Inoltre, se è implementato lo **swapping**, i processi si possono spostare dalla memoria centrale al disco per un ulteriore risparmio di spazio. Il salvataggio e ripristino dello stato della CPU si dice invece **cambio di contesto** e il tempo necessario per attuarlo varia in base alla velocità dell'architettura.

Nella maggior parte dei sistemi operativi i processi lavorano concorrentemente. Si ritiene dunque necessario fornire un meccanismo per la loro creazione e terminazione. I processi genitori possono crearne di figli, i quali a loro volta possono crearne altri. Ciò forma un albero di processi. Per poterli distinguere, i sistemi operativi come Linux li identificano tramite un **identificatore di processo** (pid), un valore numerico intero univoco per ogni processo.

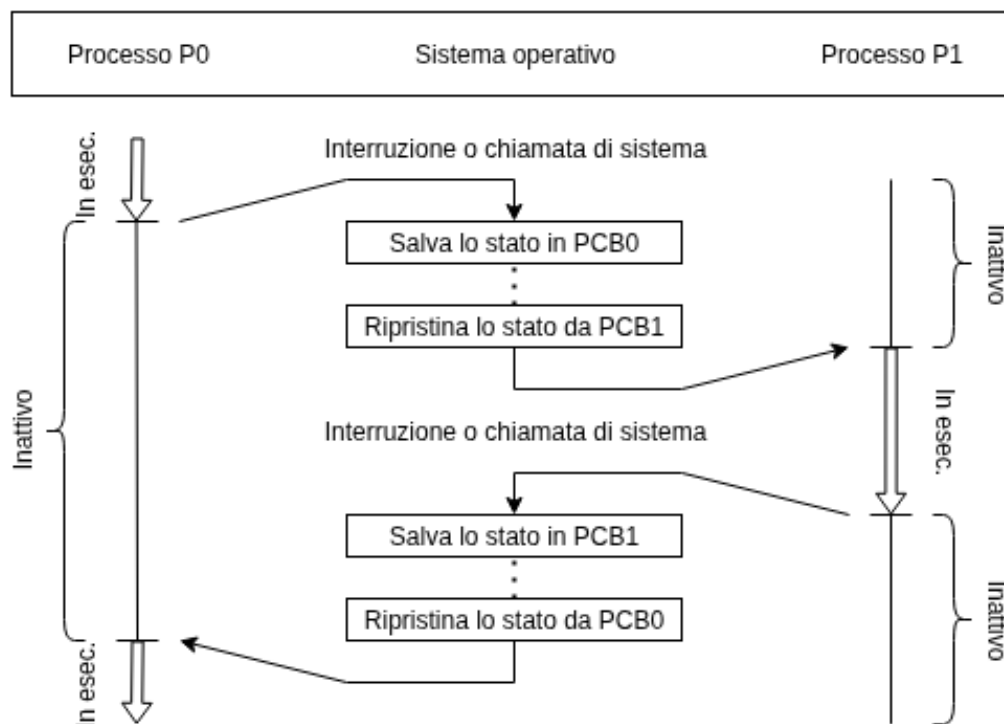


Figura 3.4: Diagramma di cambio di contesto

Generalmente, la creazione di un processo implica l'impiego di determinate risorse: ciò vale sia per genitori che figli. Questi ultimi, in particolare, possono riceverle direttamente dal sistema operativo oppure da un insieme ristretto del genitore. Quest'ultima opzione risulta preferibile in quanto limitando le risorse di un figlio si può evitare che il processo sovraccarichi il sistema usando troppa memoria.

Oltre alle varie risorse fisiche e logiche, il genitore può passare ai figli dei dati di inizializzazione, non dissimilmente da una chiamata a funzione. Possiamo avere le seguenti istanze:

- Il genitore continua l'esecuzione concorrentemente al figlio.
- Il genitore attende la terminazione di alcuni o tutti i processi figli.

Inoltre, ci sono due possibilità per quanto riguarda lo spazio di indirizzi del figlio:

- Il figlio è un duplicato del genitore e ha stessi dati e programma.
- Nel figlio si carica un nuovo programma.

Prendiamo come esempio il workflow di UNIX, dove i nuovi processi sono identificati con un pid, generati con `fork()` e composti da una copia dello spazio di indirizzi del genitore. Ciò consente una facile comunicazione tra genitore e figlio.

Entrambi i processi continuano la loro esecuzione all'istruzione successiva al `fork()`, il quale riporta il valore zero nel nuovo processo, ma ritorna il pid del figlio al genitore, per identificarlo. In genere, dopo la `fork()`, uno dei due processi impiega una `exec()` per sostituire lo spazio di memoria del processo con un nuovo programma. La chiamata carica in memoria un file binario cancellando l'immagine di memoria del programma che la contiene e avvia la sua esecuzione. Così i due processi possono comunicare e procedere in modi diversi.

Il genitore può, come detto prima, generare più figli o aspettare la loro terminazione invocando `wait()`. La sua chiamata consente di rimuovere sé stesso dalla coda dei processi pronti fino alla terminazione del figlio. Inoltre, siccome `exec()` sovrappone lo spazio di indirizzi del processo con un nuovo programma, non può restituire il controllo a meno che non si verifichi un errore. Usando la chiamata `execlp()`, il figlio sovrappone il proprio spazio di indirizzi con una stringa, per esempio `execlp("/bin/ls")`, ovvero il comando per ottenere l'elenco dei file in una directory. Quando un processo figlio termina, chiamando implicitamente o esplicitamente `exit()`, il genitore chiude la fase di attesa data da `wait()` e termina a sua volta con `exit()`.

Un processo termina quando è stata eseguita la sua ultima istruzione e inoltra la richiesta di terminazione al sistema operativo con `exit()`. Se è un figlio, riporta un'informazione di stato al genitore se ha invocato `wait()`. Tutte le risorse del processo saranno ora liberate dal sistema operativo. I processi possono essere terminati anche per altre ragioni, come un'interruzione, chiamabile solo dal padre, il quale deve necessariamente conoscere il pid del figlio per poterlo terminare. Generalmente, un genitore può porre fine alla vita di un figlio per svariati motivi, tra cui:

- Il figlio ha ecceduto nell'uso delle risorse.
- Il compito assegnato al figlio è terminato.
- Il genitore termina la propria esecuzione. In particolare, i processi terminati il cui padre è ancora attivo sono detti **zombie**, mentre se il padre termina senza invocare `wait()`, i figli saranno adottati da `systemd` per la chiusura e si dicono **orfani**.

Se un processo termina, così dovranno fare anche gli eventuali figli: questo meccanismo è detto **terminazione a cascata** ed è una procedura avviata dal sistema operativo. Non è tuttavia comune a tutti i sistemi operativi.

La terminazione fornisce uno stato di uscita, dove lo 0 indica una terminazione normale, mentre altri valori segnalano terminazioni anomale. Segue la deallocazione delle risorse utilizzate per il processo.

3.2 Comunicazione tra processi

All'interno dei sistemi operativi definiamo due tipi di processi: **indipendenti** se non influiscono né sono influenzati dagli altri e **cooperanti** quando influenzano o sono influenzati. I motivi per la creazione di un ambiente cooperativo tra i processi sono:

- **Condivisione di informazioni:** Più utenti potrebbero volere accesso alle stesse informazioni.
- **Velocizzazione del calcolo:** Alcune operazioni sono più rapide quando divise in sottoattività eseguibili in parallelo.
- **Modularità:** Suddivisione delle funzioni in processi o thread distinti.

I processi cooperanti richiedono di un meccanismo per la comunicazione; i due principali si dicono a **memoria condivisa**, richiedente una zona di memoria allocata dinamicamente condivisa dai processi, e a **scambio di messaggi**, dove la comunicazione avviene tramite l'omonimo meccanismo.

Il primo paradigma può risultare più veloce rispetto all'altro perché richiede l'intervento del kernel esclusivamente per definire la zona condivisa, mentre lo scambio di messaggi è implementato tramite chiamate di sistema.

In particolare, per i sistemi a memoria condivisa, la zona allocata si trova nello spazio degli indirizzi del processo che la riserva e richiede un accordo tra le entità che devono utilizzarla, poiché la sua gestione è fuori dal dominio del sistema operativo. Fondamentalmente è utilizzato un paradigma di **produttore-consumatore**, il quale vede un primo processo produrre informazioni che verranno consumate dal secondo.

L'esecuzione concorrente di due processi comunicanti avviene infatti grazie a un **buffer** che può essere riempito dal produttore e svuotato dal consumatore. Naturalmente deve esserci una sincronizzazione tra i due per fare in modo che non si consumino informazioni prima di crearle. Ci sono inoltre due tipi di buffer utilizzabili:

- **Illimitato:** Consente al produttore di creare sempre nuove informazioni.
- **Limitato:** Pone una dimensione al buffer condiviso; il consumatore dovrà attendere se è vuoto e il produttore dovrà attendere se è pieno.

In una comunicazione basata sullo scambio di messaggi è invece fondamentale fornire strumenti per la comunicazione e un meccanismo di sincronizzazione dei processi senza condividere lo spazio di indirizzi. Si prevede sempre l'utilizzo di almeno due operazioni primitive: `send()` per l'invio dei messaggi e `receive()` per la loro ricezione.

I messaggi possono presentarsi con lunghezza fissa o variabile e sono inviati grazie a un apposito **canale di comunicazione**, realizzabile in più modi secondo la tipologia scelta

tra **diretta/indiretta**, **sincrona/asincrona** oppure con gestione **automatica/esplícita** del buffer. Per comunicare è inoltre necessario che i processi possano fare riferimento ad altri, quindi conoscere i loro identificativi.

In una **comunicazione diretta** ogni processo che intende comunicare deve nominare esplicitamente il ricevente o trasmittente e definisce le primitive come segue:

- `send(P, message)`: Invia message al processo P.
- `receive(Q, message)`: Riceve message dal processo Q.

In uno schema tale, devono valere però alcune caratteristiche: in primo luogo, tra ogni coppia di processi comunicanti si deve instaurare automaticamente un canale ed entrambi devono conoscere solamente la reciproca identità. Il canale sarà uno solo ed è associato esclusivamente a una coppia di processi.

Una **variante** di questo modello presenta un'asimmetria nell'indirizzamento; solamente il processo che invia il messaggio nominerà il ricevente, modificando le primitive come segue:

- `send(P, message)`: Invia message al processo P.
- `receive(id, message)`: Riceve un messaggio da qualsiasi processo, identificato con id.

Entrambi questi schemi presentano una scarsissima modularità, rendendoli poco ideali se è necessario effettuare modifiche di qualche tipo. Un'idea più agevole è invece data dalla **comunicazione indiretta**, la quale impiega delle **porte** o **mailbox** univocamente identificate per la ricezione dei messaggi. In un modello tale, i processi possono comunicare esclusivamente se condividono una cassetta e modifica le primitive in questo modo:

- `send(A, message)`: Invia message alla mailbox A.
- `receive(A, message)`: Riceve message dalla mailbox A.

Ne consegue che un canale di comunicazione può esistere solamente se due o più processi condividono una mailbox, ma ciò rende i canali non più esclusivi a soli due processi. Tra ogni coppia possono infatti essercene diversi, ognuno dei quali corrispondente a una mailbox.

Dando l'accesso a più di due processi ad una sola mailbox si introduce il problema della decisione del destinatario del messaggio. Non c'è una dinamica deterministica per la scelta del processo ricevente, quindi si sceglie di decidere arbitrariamente con una **cablatura**¹ oppure di scrivere un algoritmo per la selezione del destinatario, come quello

¹Le cablature, in inglese **hard-coding**, sono generalmente considerate cattiva prassi.

di **round robin**, che assegna i messaggi ai processi ciclicamente, e comunicare l'identità del ricevente al trasmittente.

Una mailbox può appartenere sia a processi che al sistema operativo; nel primo caso, si trova nello spazio di indirizzi del processo e occorre distinguere tra proprietario e utente. Quando termina, la relativa casella scompare ed è inviato un messaggio a tutti gli altri processi in essa comunicanti.

Quando la mailbox è invece posseduta dal sistema operativo, presenta una vita autonoma ed è quindi indipendente da altri processi. Il sistema offre inoltre un meccanismo per la creazione e rimozione di mailbox, come anche la ricezione e invio di messaggi. Quando un processo crea una mailbox con il sistema operativo, ne è il proprietario predefinito, ma è possibile modificarne la proprietà con apposite chiamate di sistema.

Come già menzionato, lo scambio di messaggi può avvenire tramite due modalità: **sincrona**², dove il mittente si blocca finché il messaggio non è inviato ed il destinatario si sospende finché non è disponibile un messaggio, e **asincrona**, che vede le due entità continuare la propria esecuzione dopo invio o ricezione del messaggio. È inoltre possibile avere più combinazioni di primitive in quanto non sono mutualmente esclusive. Infine, quando la comunicazione è diretta o indiretta, i messaggi scambiati risiedono in **code temporanee**, delle quali esistono tre tipi:

- **Capacità zero:** Il canale di comunicazione non può ospitare messaggi; il trasmittente si blocca finché il ricevente non è pronto a ricevere il messaggio.
- **Capacità limitata:** La coda può ospitare un numero n di messaggi. Quando questa è piena, il trasmittente blocca la sua esecuzione e attende l'invio del messaggio, altrimenti invia il contenuto senza fermarsi.
- **Capacità illimitata:** La coda non ha limiti di spazio; il trasmittente non si ferma mai.

²Se mittente e destinatario hanno comportamento sincrono, il loro incontro è detto **rendez-vous**.

Capitolo 4

Thread e concorrenza

4.1 Programmazione multicore e multithreading

Definiamo un **thread** come l'unità di base di uso della CPU, composto da un relativo identificatore, un program counter, un insieme di registri e una stack di attivazione. Condivide con altri thread appartenenti allo stesso processo le sezioni di codice, dati e altre risorse, come per esempio i file aperti.

Finora abbiamo parlato di **heavyweight processes**, ovvero quelli composti da un solo thread, mentre in questa sezione discuteremo degli scopi e dei vantaggi della programmazione di processi **multithreaded**.

La maggior parte delle applicazioni odierne è multithreaded; si codificano infatti come un processo a sé stante che comprende più thread di controllo. Per esempio, un web browser ha un filo per rappresentare a schermo le pagine e un altro per scaricare i dati dalla rete. La ragione principale per la quale si è passati da un paradigma monolitico al multithread è che sempre più spesso era necessario per le applicazioni saper gestire più compiti allo stesso tempo; un web server intensamente utilizzato vede molti client che vi accedono concorrentemente, per esempio.

Una prima soluzione a questo problema sarebbe di creare un nuovo processo per ogni richiesta, ma risulterebbe estremamente oneroso in termini di risorse e tempo. È nettamente più conveniente creare un thread apposito per ogni nuova richiesta, poiché condividendo le sezioni non è necessario allocare nuova memoria.

Anche la maggior parte dei kernel dei sistemi operativi moderni è multithreaded; all'avvio di un sistema Linux vengono infatti creati diversi thread per le varie attività da svolgere, come la gestione dei dispositivi o della memoria. Determinati algoritmi possono inoltre trarre vantaggio da questo paradigma riducendo drasticamente i tempi di risposta.

Possiamo raggruppare i vantaggi del paradigma multithread in quattro categorie principali: in primo luogo, si riducono i **tempi di risposta**, perché consentiamo al programma di continuare la sua esecuzione mentre gestisce altre operazioni su diversi thread.

Richiamando come i processi comunicano, ovvero tramite memoria condivisa o scambio di messaggi, ai thread è possibile **condividere risorse** e memoria del processo a cui appartengono.

Come già menzionato, la creazione di nuovi processi risulta più costosa rispetto alla generazione di nuovi thread, perché per questi ultimi si richiede l'impiego di meno memoria, tempo, ed è anche più semplice gestirne il cambio di contesto.

Infine, i processi multithreaded sono facilmente **scalabili**; nelle architetture multiprocessore i thread si possono eseguire in parallelo, incrementando il parallelismo.

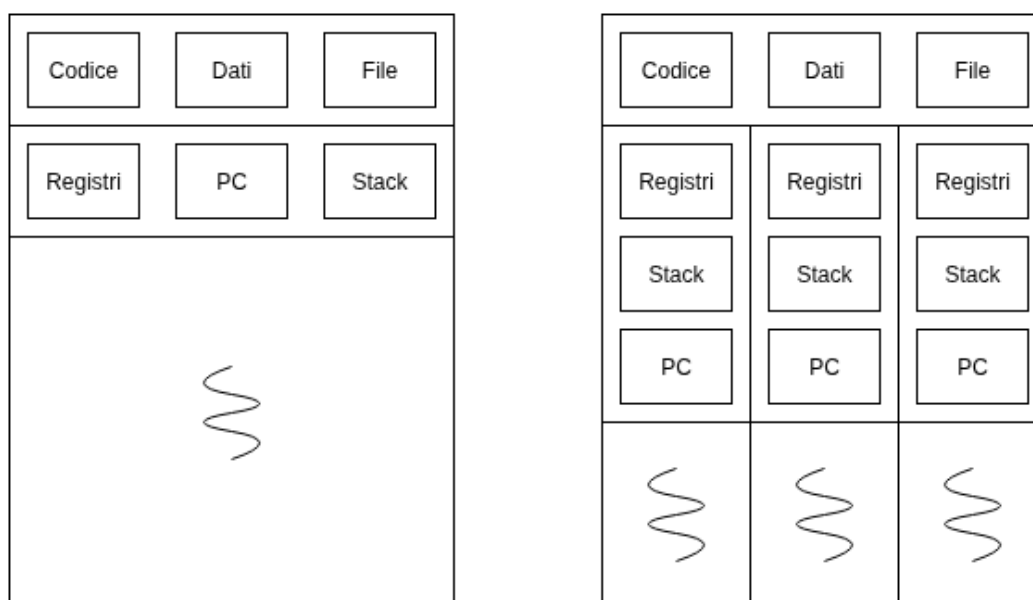


Figura 4.1: Panoramica processi a singolo thread e multithreaded

In risposta alle necessità di maggiore potenza di calcolo, i calcolatori odierni presentano architetture multiprocessore, quindi che dispongono di più core di elaborazione su un unico chip. La **programmazione multithread** definisce un meccanismo per l'utilizzo efficiente di queste architetture, sfruttando al meglio la concorrenza.

Quando parliamo di **esecuzione concorrente** in questo contesto, intendiamo che i thread possono funzionare in parallelo perché il sistema può assegnare thread diversi ad ogni core della CPU. Inoltre distinguiamo **sistema concorrente**, ovvero che supporta più task permettendo a ciascuno di progredire nell'esecuzione, da **sistema parallelo**, dove è effettivamente possibile eseguire simultaneamente più compiti.

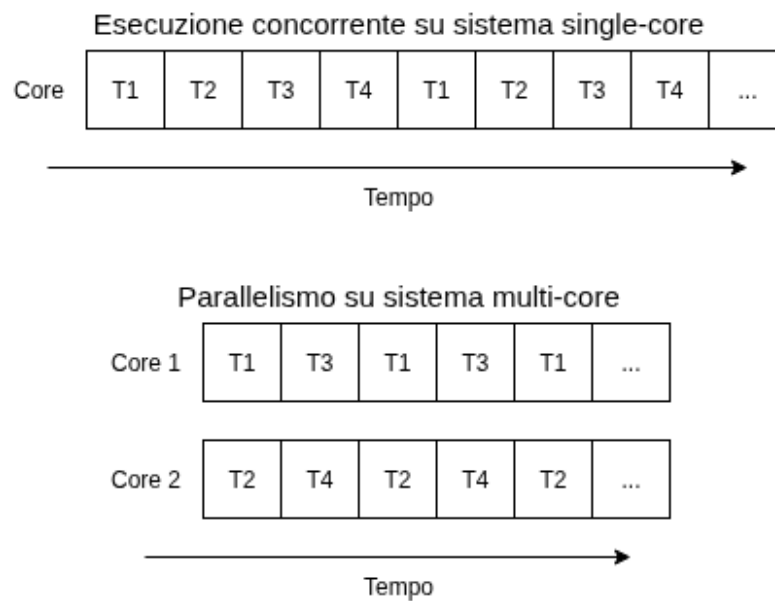


Figura 4.2: Panoramica concorrenza e parallelismo

Questo cambio di forma porta delle nuove sfide in ambito progettuale e di sviluppo applicazioni; chi crea e mantiene sistemi operativi dovrà scrivere algoritmi di scheduling orientati all'utilizzo di più core e i programmatori di applicazioni dovranno renderle multithreaded. In particolare, identifichiamo le seguenti sfide nella programmazione multicore:

- **Identificazione compiti:** Necessità di individuare aree separabili in task e quindi in thread.
- **Bilanciamento:** Le task devono eseguire compiti di mole e valore confrontabili. Se presentano un basso valore computazionale, potrebbe non valerne la pena di dedicarvi un intero core.
- **Suddivisione dati:** Definire i dati manipolati e a cui accedono le task. Vanno suddivisi per essere usati su core distinti.
- **Dipendenze dati:** In caso di dipendenze tra due o più task è necessario anche implementare un meccanismo di sincronizzazione per garantirne il corretto flusso di esecuzione.
- **Test e debugging:** Avendo più fili di esecuzione risulta più difficile fare testing e debugging.

Definiamo inoltre due tipi di parallelismo: quello dei **dati**, che riguarda la distribuzione di sottoinsiemi di dati su più core e l'esecuzione di una stessa operazione su ognuno di essi, e il **parallelismo delle attività**, che distribuisce thread su più core, i quali possono operare sugli stesso dati o diversi. Questi tipi di parallelismo non sono mutualmente esclusivi; infatti nei sistemi operativi è comune trovarne un'implementazione ibrida.

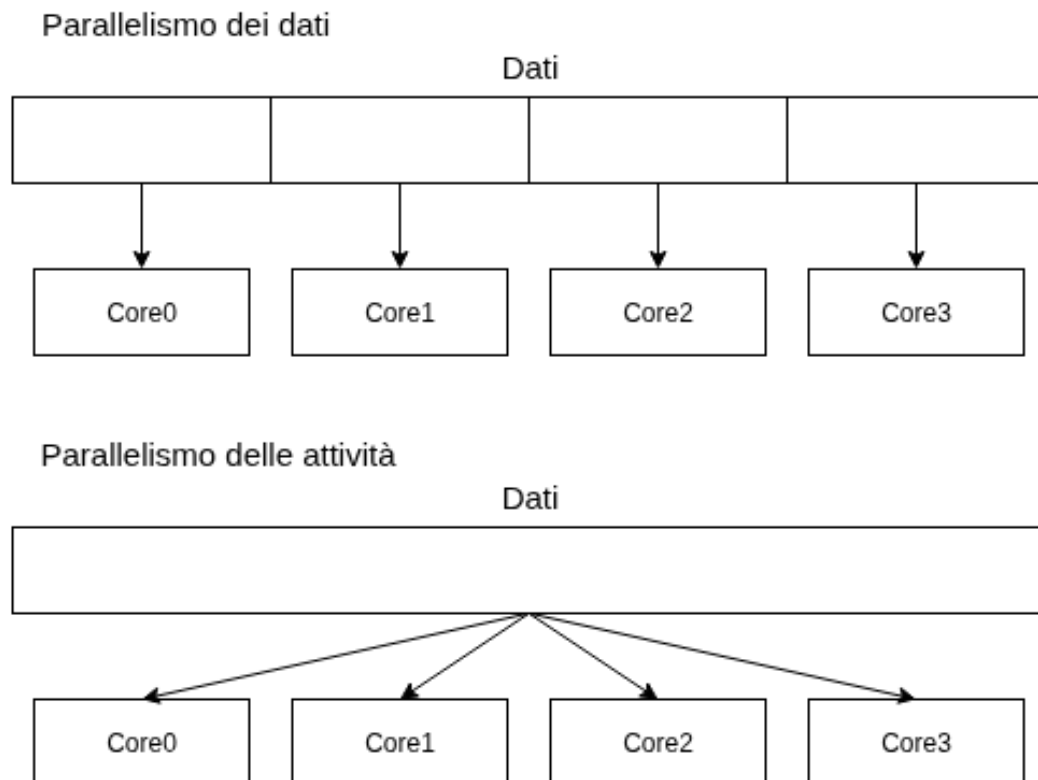


Figura 4.3: Tipi di parallelismo

Distinguiamo i thread su due livelli: a **livello utente** sono gestiti sopra quello del kernel e senza il suo supporto, mentre a **livello kernel** sono gestiti direttamente dal sistema operativo. È necessario che questi siano messi in relazione; esistono tre modelli principali per implementarla:

- **Modello da molti a uno:** Fa corrispondere più thread utente ad un solo thread kernel. Si tratta di una gestione efficiente dei thread effettuata grazie ad un'apposita libreria nello spazio utente. Tuttavia, se il thread kernel invoca una system call bloccante, bloccherà l'intero processo. Inoltre, dato che un solo thread alla volta può accedere al kernel, è impossibile eseguire thread multipli in parallelo in sistemi multicore. Per queste ragioni è un modello raramente usato.

- **Modello da uno a uno:** Mette in corrispondenza ogni thread utente con un thread kernel, offrendo un maggiore grado di concorrenza e permettendo l'esecuzione in parallelo nei sistemi multiprocessore. Tuttavia, l'esistenza di un thread utente richiede quella del corrispondente thread kernel; essendo che questo meccanismo può sovraccaricare le prestazioni delle applicazioni, si sceglie di porre un limite al numero di thread supportabili, ma grazie alle architetture moderne si è potuto ridurre questa restrizione.
- **Modello da molti a molti:** Mette in corrispondenza più thread utente a un numero minore o uguale di thread kernel, dove il numero di thread kernel può essere calibrato in base all'applicazione o all'architettura del sistema. Per quanto riguarda la concorrenza, consente ai programmatori di creare liberamente un numero arbitrario di thread e i corrispondenti thread kernel si possono eseguire in parallelo nelle architetture multicore, superando gli svantaggi dei due precedenti modelli.

Una variante di questo modello è il **modello a due livelli**, che mantiene la corrispondenza fra più thread utente e un numero minore o uguale di thread kernel, ma consente anche di vincolare un primo con un secondo, come per il modello uno a uno. Si tratta del modello più flessibile di tutti, ma è anche quello più difficile da implementare nella pratica.

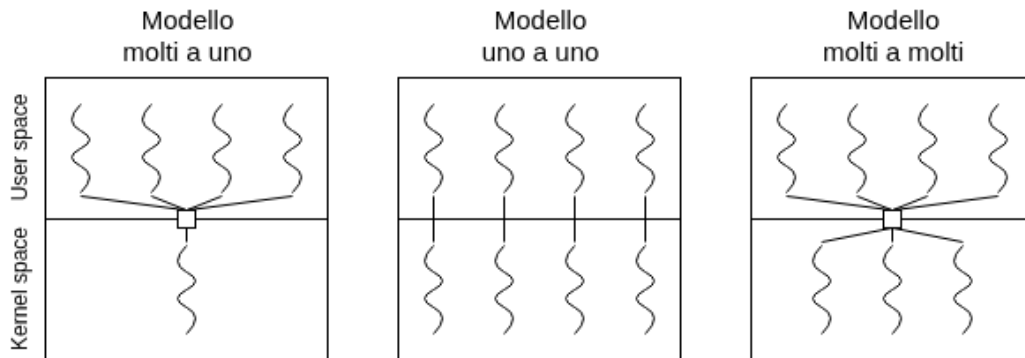


Figura 4.4: Modelli di supporto multithreading

4.2 Librerie dei thread

Le librerie dei thread forniscono ai programmatori una API per la loro creazione e gestione. Ci sono due metodi principali per l'implementazione di thread: collocare la **libreria a livello utente**, vedendo codice e strutture dati per la libreria risiedere nello spazio

utente; invocare una funzione implica una chiamata locale sul livello utente. Alternativamente è possibile collocare la **libreria a livello kernel**, facendo risiedere le risorse nel relativo spazio e corrispondere l'invocazione di funzioni con le chiamate di sistema al kernel.

La libreria utilizzata in questo corso è **Pthreads**, di POSIX, la quale può essere realizzata in entrambe le modalità appena discusse. Con questa API, tutti i dati dichiarati a livello globale sono condivisi tra tutti i thread generati dai programmi, mentre quelli a livello locale sono memorizzati nello stack proprietario del filo, dando quindi ad ogni thread una propria copia dei dati locali.

Esistono naturalmente più strategie per l'implementazione di thread: il **threading asincrono** vede il genitore creare un figlio e continuare la sua esecuzione indipendentemente dal workflow del figlio, usato normalmente quando è richiesta una scarsa condivisione dei dati. Il secondo metodo, invece, detto **threading sincrono**, blocca l'esecuzione del processo padre fino al termine dell'esecuzione dei figli¹; generalmente è implementato quando vi è una significativa necessità di condivisione dati.

L'API fornito da Pthreads ci consente di implementare questi meccanismi in linguaggio C; sarà necessario includere la relativa libreria e usare le funzioni apposite per inizializzare, creare e terminare thread di esecuzione. Prendiamo il seguente esempio:

```
1  # include <pthread.h>
2  # include <stdio.h>
3  # include <stdlib.h>
4
5  int sum; // Variabile globale, condivisa dai thread in questo file
6  void *runner(void *param); // Chiamata dai thread
7
8  int main (int argc, char *argv[]) {
9      pthread_t tid; // Identificatore del thread
10     pthread_attr_t attr; // Insieme di attributi del thread
11
12     // Inizializza thread con attributi predefiniti
13     pthread_attr_init(&attr);
14     // Crea il thread
15     pthread_create(&tid, &attr, runner, argv[1]);
16     // Attende terminazione thread figlio
17     pthread_join(tid, NULL);
18
19     printf("%d\n", sum);
20 }
21
22 // Thread eseguito nella seguente funzione runner
23 void *runner (void *param) {
```

¹Questa strategia è chiamata anche **fork-join**, perché il genitore attende che tutti i figli si ricongiungano col padre prima di riprendere la sua esecuzione.

```
24     int upper = atoi(param);
25     sum = 0;
26
27     for (int i = 1; i <= upper; i++) sum += i;
28
29     pthread_exit(0);
30 }
```

Il programma implementa un semplice workflow dove un processo padre genera un thread figlio, nel quale si ritorna la somma di tutti i numeri fino al parametro passato in funzione.

Per creare un thread figlio, il padre deve in primo luogo assegnargli un identificativo e degli attributi, inizializzati tramite `pthread_attr_init()`. Dopodiché potrà generare il figlio con `pthread_create()`, che prende in input questi due valori, la funzione dove è eseguito il thread e il parametro ivi indicato. Per garantire un workflow sincrono, il padre usa `pthread_join()` per fermare la sua esecuzione fin quando tutti i figli non hanno terminato il loro lavoro.

È possibile naturalmente generare più thread; una strategia comune per la modalità sincrona è di implementare un ciclo for e un counter apposito che si incrementa fino al termine di ogni figlio.

4.3 Threading implicito

Con la costante evoluzione della programmazione multicore si richiede anche la gestione di centinaia o migliaia di thread, comportando l'attuazione di strategie per la progettazione delle applicazioni parallele o concorrenti. Una prima idea, detta **threading implicito**, vede il trasferimento della creazione e gestione del threading ai compilatori e alle librerie. Richiede agli sviluppatori l'identificazione dei task da eseguire in parallelo, i quali saranno codificati come funzioni.

Rimane tuttavia il problema del consumo delle risorse in base al numero di thread attivi. La soluzione è data dall'impiego dei **gruppi di thread**, quindi la creazione di un determinato numero di fili organizzati in gruppi all'attivazione del processo. La pool contiene i thread pronti ad eseguire richieste; quando si attiva un task, viene prelevato un filo da questo gruppo e gli si assegna il compito. Esaudita la richiesta, potrà essere reinserito nella pool. Questa strategia porta diversi vantaggi:

- Fornisce un servizio più rapido rimuovendo il ritardo dovuto alla creazione dinamica del thread.
- Limitando il numero di thread esistenti, consente l'esecuzione del programma anche su sistemi low-specs.
- Separare il task dalla meccanica della sua creazione dona modularità al sistema.

In particolare, il numero di thread massimo si può determinare in base ad alcuni fattori come il numero di core della CPU o la quantità di memoria fisica disponibile.

La strategia di **fork-join** trattata nelle sezioni precedenti è un'altra modalità per l'implementazione di threading implicito; si possono definire dei task paralleli la cui assegnazione ai thread è gestita dalla libreria di runtime. Questa è di fatto una realizzazione di un thread pool sincronizzato in cui il processo padre attende il completamento di tutti i task prima di proseguire.

4.4 Problemi di gestione multithread

Come già discusso, la programmazione multithread e multicore include la gestione di diversi problemi. In primo luogo, se un thread invoca `fork()`, il nuovo processo potrebbe contenere un duplicato di tutti i fili oppure del thread invocante, oppure, invocando `exec()`, il programma specificato come parametro sostituirebbe l'intero processo chiamante insieme a tutti i suoi thread. Si necessita di un meccanismo per un controllo corretto.

Nei sistemi UNIX si usano i **segnali** per comunicare ai processi il verificarsi degli eventi; questi possono essere ricevuti in modalità sincrona o asincrona, ma seguono entrambi lo stesso schema di generazione, invio a processo e gestione.

Per esempio, un programma potrebbe tentare un accesso illegale su alcune zone di memoria; in tal caso è inviato un **segnale sincrono** al processo colpevole e si procederà con la sua terminazione. Quando un segnale è invece causato da un evento esterno al processo, questo lo riceve in modalità **asincrona**. Generalmente i segnali si possono gestire in due modi: mediante un gestore **predefinito** del kernel oppure con uno definito dall'utente. In particolare, la gestione può ignorare il segnale oppure richiedere la terminazione del processo, ma nei processi multithread è necessario capire a quale thread bisogna inviarlo. Generalmente, esistono le seguenti istanze:

- Segnale inviato al thread al quale si riferisce.
- Segnale inviato ad ogni thread del processo.
- Segnale inviato a specifici thread del processo.
- Definizione di un thread specifico per ricevere i segnali.

Inoltre, il modo per recapitare il segnale dipende dal suo tipo; quelli sincroni vanno inviati al thread che ha generato l'evento, mentre per gli asincroni può dipendere dal contesto. Per esempio, il comando `bash` per terminare i programmi `Ctrl+C` produce un segnale inviato ad ogni thread del processo in esecuzione. La funzione standard per inviare un segnale nei sistemi UNIX è:

```
kill(pid_t pid, int signal)
```

La quale prende come parametri l'identificatore del processo (pid) al quale deve essere recapitato il particolare segnale (signal). La funzione equivalente per Pthreads è invece:

```
pthread_kill(pthread_t tid, int signal)
```

Che richiede gli stessi parametri della precedente. Essendo che i segnali devono essere gestiti una sola volta, questi sono generalmente inviati al primo thread che non li blocca.

Consideriamo ora la **cancellazione dei thread**, l'operazione che permette la terminazione di un filo prima del completamento del suo compito. Il thread da cancellare si dice **bersaglio** e la sua terminazione può avvenire in due modalità: **asincrona**, dove un thread fa immediatamente terminare il target, e **differita**, che vede il bersaglio controllare periodicamente la presenza del segnale di un altro thread, il quale comunica se deve essere terminato o meno. In particolare, possono presentarsi problemi con la cancellazione quando ci sono risorse assegnate a un thread cancellato oppure se si termina un thread mentre sta aggiornando dati condivisi con altri fili. Quest'ultimo caso, se trattato con cancellazione asincrona, risulta particolarmente problematico perché il sistema operativo potrebbe non recuperare tutte le risorse ad esso assegnate.

Nell'API di Pthreads, la cancellazione si avvia tramite la funzione **pthread_cancel()**, dove viene passato come parametro il thread id del filo da terminare. Segue esempio:

```
1 pthread_t tid;
2
3 // Crea il thread
4 pthread_create(&tid, 0, worker, NULL);
5
6 // Blocco di codice intermedio
7
8 // Cancella il thread
9 pthread_cancel(tid);
10
11 // Attende la terminazione del thread
12 pthread_join(tid, NULL)
```

Notare che la chiamata a `pthread_cancel()` è solamente la richiesta di cancellazione, perché l'effettiva terminazione dipende in base alle impostazioni per la gestione di terminazioni del thread di destinazione. In particolare, si può impostare uno stato di cancellazione e il suo tipo usando una API, scegliendo tra le seguenti opzioni:

Modalità	Stato	Tipo
Off	Disabilitato	-
Differita	Abilitato	Differito
Asincrona	Abilitato	Asincrono

Il tipo di cancellazione predefinito è il differito, secondo il quale la cancellazione avviene esclusivamente quando il thread raggiunge un **punto di cancellazione**. La stragrande maggioranza delle chiamate di sistema bloccanti è vista come punto di cancellazione; una tecnica per crearne uno su Pthreads è tramite la funzione **pthread_testcancel()**. Effettuata la richiesta di cancellazione, è chiamata una funzione detta **gestore della pulizia**, la quale rilascia ogni risorsa legata al thread per poi eliminarlo.

Un altro dei vantaggi discussi della programmazione multithread è dato dalla condivisione dei dati tra i thread di uno stesso processo, ma può capitare che sia richiesta una copia privata di certi dati, detti **dati specifici del thread** (TLS). Questi sono definiti come statici all'interno del codice e sono quindi unici e visibili all'interno dell'intero file ove sono scritti. Pthreads include il tipo **pthread_key_t** che fornisce una chiave specifica per ogni filo, la quale può essere utilizzata per accedere ai dati TLS.

Un'ultima questione da affrontare riguarda poi la comunicazione tra la libreria del kernel e quella per i thread, necessaria nel modello da molti a molti e a due livelli. Questa struttura rende il numero di thread modificabile dinamicamente con lo scopo di ottenere prestazioni migliori e vede il collocamento di una struttura dati intermedia tra i thread kernel e utente. Questa, detta **processo leggero** (LWP) svolge una funzione di processore virtuale e ne esiste uno per ogni thread kernel. L'applicazione potrà utilizzare gli LWP per richiedere lo scheduling di un thread a livello utente. Tuttavia, essendo collegati, se un thread kernel si blocca, così faranno anche l'LWP e il thread utente; per questa ragione, con lo scopo di garantire un'esecuzione efficiente, è consigliato utilizzare un LWP per ogni chiamata di sistema concorrente bloccante.

In questo contesto, uno dei modelli di comunicazione tra libreria utente e kernel è detto **attivazione dello scheduler**; il quale attua la seguente procedura:

1. Il kernel fornisce alle applicazioni gli LWP sui quali scheduleranno i thread utente. Segnerà inoltre la presenza di eventi tramite la procedura di **upcall**².
2. Le upcall sono gestite dalla libreria dei thread mediante un gestore eseguito su LWP. Si identifica il thread indicato nella chiamata.
3. Il kernel assegna all'applicazione un nuovo LWP, la quale eseguirà un gestore upcall su di esso.
4. Il gestore salva lo stato del thread bloccante e rilascia l'LWP sul quale stava venendo eseguito. Pianifica inoltre l'esecuzione di un altro thread sul nuovo LWP.

²Un'istanza per innescarne una è per esempio quando un processo è sul punto di bloccarsi

5. Al verificarsi dell'evento atteso dal thread bloccante, precedentemente segnalato dal kernel, questo effettua una nuova upcall³ alla libreria per comunicare che il thread bloccato è nuovamente disponibile per l'esecuzione.
6. L'applicazione contrassegna il thread in questione come pronto ed esegue lo scheduling di uno dei fili appartenenti a questo gruppo su uno dei processori virtuali disponibili.

Discutiamo adesso del funzionamento dei thread in ambiente Linux. Questo sistema operativo non distingue tra processi e thread e impiega per entrambi il termine **task**. La chiamata `fork()` duplica un processo e `clone()` crea un nuovo thread; quest'ultima in particolare riceve come parametro alcuni flag per stabilire le risorse del padre da condividere col figlio; questi sono:

- **CLONE_FS**: Condivisione informazioni sul file system.
- **CLONE_VM**: Condivisione dello stesso spazio di memoria.
- **CLONE_SIGHAND**: Condivisione dei gestori dei segnali.
- **CLONE_FILES**: Condivisione dei file aperti.

Questa condivisione a intensità variabile è possibile grazie al modo in cui i task sono strutturati nel kernel. Esiste infatti un'unica struttura dati che si serve di puntatori ad altri tipi struct contenenti i dati effettivi. In questi termini, `fork()` crea un nuovo task insieme con una copia di tutte le strutture dati del genitore, mentre `clone()` fa puntare il nuovo task alle strutture dati del genitore in base ai flag passati nella funzione. Questa funzione può essere utilizzata anche per l'implementazione di **container**, una tecnica di virtualizzazione fornita dal sistema operativo per la creazione di diversi sistemi Linux.

³Naturalmente, anche questa chiamata necessita di un LWP, seguendo il procedimento appena descritto.

Capitolo 5

Scheduling della CPU

5.1 Concetti di base

Ribadiamo che lo scopo della programmazione multicore è mantenere la CPU quanto più attiva possibile, minimizzando i tempi in cui non sta elaborando processi. Questo concetto, nei sistemi multicore, è esteso ad ogni core di elaborazione del sistema, la cui gestione avviene tramite **scheduling**.

I processi in esecuzione si compongono di un **ciclo di elaborazione CPU** e un **ciclo di attesa di completamento operazioni I/O**. L'esecuzione ha inizio con una sequenza di operazioni CPU, detta **CPU burst**, seguite da operazioni I/O, chiamate **I/O burst**; questi due elementi si avvicendano fino al termine del processo con la chiamata opportuna al sistema operativo.

Dove le durate dei CPU burst variano in base al tipo di processo e all'architettura, in genere, i processi I/O-bound presentano CPU burst di breve durata, mentre i CPU-bound ne mostrano di lunga durata.

Ogni volta che la CPU passa in stato di inattività, è compito dello **scheduler a breve termine** selezionare il processo dalla ready queue e dargli il controllo dell'unità. Le queues non sono necessariamente code FIFO; possono essere implementate anche con logica LIFO o altre strutture dati come alberi o liste concatenate; tuttavia, gli elementi al suo interno devono sempre essere i PCB dei processi. Le decisioni per lo scheduling sono effettuate in quattro contesti:

- Quando un processo passa da esecuzione ad attesa, come per le richieste I/O oppure quando si invoca `wait()`.
- Quando un processo passa da esecuzione a pronto, come quando si segnala un interrupt.
- Quando un processo passa da attesa a pronto, come quando si è completata un'operazione I/O.

- Quando un processo termina l'esecuzione.

Quando lo scheduler interviene nei casi 1, 4 si dice che è **senza prelazione** e implica che quando si assegna il controllo della CPU a un processo, questo lo mantiene fino al momento del rilascio, dovuto al suo termine o al passaggio in stato di attesa.

Negli altri due casi, lo schema di scheduling è **con prelazione**; sebbene sia la strategia attuata da tutti i moderni sistemi operativi, presenta alcuni svantaggi, come portare a race condition¹ quando i dati sono condivisi tra i processi, dinamica valida anche per il kernel. Si potrebbe pensare quindi di scegliere uno schema senza prelazione, ma non è adeguato per le applicazioni in tempo reale. È quindi necessario applicare meccanismi per prevenire la formazione di race condition.

Un altro elemento coinvolto nello scheduling è il **dispatcher**, il modulo che passa effettivamente il controllo della CPU al processo scelto dallo scheduler. Le mansioni comprendono il context switch da un processo all'altro, il passaggio alla user-mode e il salto alla posizione adeguata dei programmi utente per la loro esecuzione.

Idealmente, i dispatcher dovrebbero essere quanto più ottimizzati possibile poiché i cambi di contesto avvengono frequentemente. Chiamiamo **latenza di dispatch** il tempo richiesto per fermare un processo e avviare un altro cedendo il controllo della CPU. In particolare, esistono due tipi di context switch: **volontario**, se un processo cede il controllo della CPU in seguito alla richiesta di una risorsa attualmente non disponibile, e **involontario** quando il comando è sottratto dal processo, per esempio quando è esercitata la prelazione.

Esistono più algoritmi di scheduling con diverse caratteristiche, ed è necessario prenderle in considerazione prima di implementare quello più adatto al contesto in esame. Consideriamo alcuni criteri per una scelta consapevole:

- **Utilizzo della CPU:** L'unità deve essere quanto più attiva possibile e il suo utilizzo si misura in percentuale. In un sistema a basso carico è di circa 40%, mentre se è ad alto carico, si aggira sul 90%.
- **Throughput:** Numero di processi completati nell'unità di tempo.
- **Tempo di completamento:** Intervallo di tempo che intercorre tra sottomissione del processo al suo completamento. È il risultato della somma dei tempi di attesa per l'inserimento in memoria, nella ready queue, durante l'esecuzione e nelle operazioni di I/O.
- **Tempo di attesa:** Somma dei tempi passati nella ready queue.
- **Tempo di risposta:** Tempo necessario per iniziare la risposta ad una richiesta.

¹Un processo potrebbe modificare dati che un altro in attesa deve leggere. Il risultato è non deterministico.

Generalmente, è una buona prassi cercare di massimizzare i tempi di utilizzo della CPU e minimizzare quelli di attesa, risposta e completamento. In particolare, con sistemi interattivi come quelli desktop, è auspicabile ridurre al minimo la varianza del tempo di risposta per garantire un'esperienza stabile ed efficace.

5.2 Algoritmi di scheduling

Il più semplice algoritmo di scheduling della CPU è quello in base all'ordine di arrivo: **First-come, First-served** (FCFS), il quale richiede l'implementazione della ready queue come una coda FIFO, in tal modo che lo scheduler possa recuperare il PCB del processo dalla cima della coda e assegnargli la CPU fino al termine della sua esecuzione.

Si tratta di un algoritmo senza prelazione il cui tempo medio di attesa è spesso abbastanza lungo; consideriamo infatti le seguenti coppie (processo, durata sequenza):

$$(P_1, 24ms) \quad (P_2, 3ms) \quad (P_3, 3ms)$$

Il tempo di attesa per il primo processo è $0ms$, per il secondo è $24ms$ e per il terzo $27ms$, facendo risultare come tempo medio:

$$\bar{t} = (0 + 24 + 27)/3 = 17ms$$

È chiaro come nell'eventualità di un processo esoso con una lunga durata della sequenza di istruzioni si crei un **effetto convoglio**, dove tutti i processi nella ready queue sono in attesa della terminazione del processo in esecuzione. Per questa ragione è sconsigliata la sua implementazione in sistemi con time-sharing.

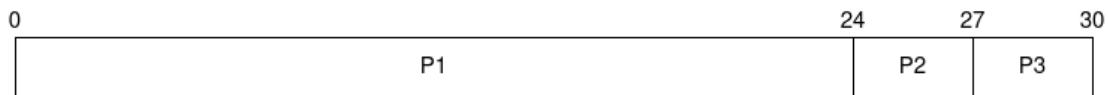


Figura 5.1: Diagramma di Gantt per algoritmo FCFS

Un secondo algoritmo è quello di scheduling per brevità: **Shortest job first** (SJF), il quale associa a ogni processo la lunghezza della sequenza di operazioni CPU successiva. Quando è disponibile, si assegna al core la più breve lunghezza; a parità di valori, invece, è assegnato con logica FCFS. Supponiamo i processi:

$$(P_1, 6ms) \quad (P_2, 8ms) \quad (P_3, 7ms) \quad (P_4, 3ms)$$

Verranno eseguiti in base alla loro lunghezza, quindi nell'ordine: P_4, P_1, P_3, P_2 , facendo risultare un tempo di attesa medio di:

$$\bar{t} = (0 + 3 + 9 + 16)/4 = 7ms$$

SJF è un algoritmo ottimale rispetto al tempo medio di attesa, tuttavia non è realizzabile a livello dello scheduling della CPU a breve termine, perché non è possibile conoscere la lunghezza della sequenza successiva. Possiamo tuttavia provare a predirlo; calcolando un valore approssimato della lunghezza si può scegliere il processo con quella più breve stimata.

La stima avviene calcolando la media esponenziale delle lunghezze misurate dalle sequenze precedenti tramite la formula:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Dove t_n è la lunghezza della sequenza n , τ_{n+1} il valore previsto per la sequenza successiva e $0 \leq \alpha \leq 1$ controlla il peso sulla predizione della storia recente e passata. Infatti, con $\alpha = 0$, la storia recente non ha effetto, mentre con $\alpha = 1$ ha significato solo la più recente sequenza di operazioni della CPU. Generalmente, questo valore corrisponde a 0,5.

L'algoritmo SJF può essere con o senza prelazione; la scelta si compie quando un nuovo processo si accoda alla ready queue mentre un altro è in esecuzione. L'implementazione con prelazione riprende il controllo del core dal processo attivo se quello nuovo risulta più breve, mentre quello senza prelazione lascia lo lascia terminare prima di riassegnare il core.

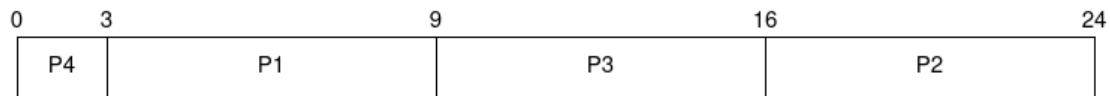


Figura 5.2: Diagramma di Gantt per algoritmo SJF senza prelazione

Il terzo algoritmo proposto riguarda lo scheduling circolare **Round-Robin** (RR), che vede fornita ad ogni processo una quantità di tempo fissa stabilita dal sistema operativo, detta **quanto**, entro la quale possono continuare la loro esecuzione.

La ready queue è implementata con una coda FIFO; lo scheduler imposta un timer per segnalare la fine di ogni quanto di tempo, inviare un segnale di interruzione e cambiare il contesto facendo riassegnare il core dal dispatcher. Se il processo non termina entro l'intervallo, è reinserito nella ready queue. Supponiamo i processi:

$$(P_1, 24ms) \quad (P_2, 3ms) \quad (P_3, 3ms)$$

Considerando un quanto di tempo di $4ms$, si avrebbe un tempo di attesa per P_1 uguale a $(10 - 4)ms$, perché nei primi $10ms$ ha potuto lavorare nel suo primo quanto e l'attesa prima di riprendere l'esecuzione è durata $6ms$, $4ms$ per P_2 e $7ms$ per P_3 , per poi continuare ad eseguire P_1 fino al suo termine. Risultato:

$$\bar{t} = (6 + 4 + 7)/3 = 5,66ms$$

I tempi di attesa e di completamento dipendono tuttavia anche dalla dimensione del quanto di tempo stabilita; se è molto grande, l'algoritmo di round-robin si riduce ad un FCFS, mentre se si sceglie un quanto molto piccolo, l'overhead dei context switch diventa dominante, degradando come diretta conseguenza le prestazioni. È quindi necessario trovare un compromesso che ottimizzi l'esecuzione senza far attendere troppo gli altri processi.

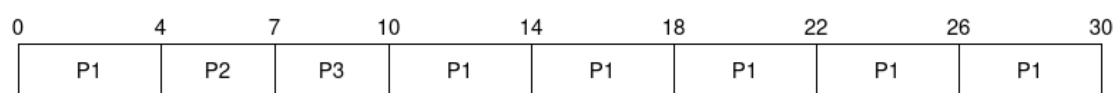


Figura 5.3: Diagramma di Gantt per algoritmo RR

L'algoritmo di scheduling con **priorità** è un caso più generale di SJF, che vede associato ad ogni processo un valore di priorità, in base al quale sarà assegnato il core di esecuzione. Le priorità sono indicate con un intervallo di numeri il cui significato è arbitrario e va deciso durante l'implementazione. In questo contesto useremo valori più bassi per indicare priorità più alte.

L'algoritmo può essere sia con prelazione che senza; nel primo caso sottrae forzatamente il core al processo in essere quando nella ready queue se ne inserisce uno nuovo di priorità più alta, mentre nel secondo si pone semplicemente in cima alla coda il nuovo processo, il quale sarà eseguito dopo la terminazione di quello attivo.

Lo scheduling con priorità porta tuttavia alcuni problemi, come quello dell'attesa indefinita (**starvation**) dei processi a bassa priorità, perché saranno potenzialmente ignorati in favore di quelli con valori più alti. Una soluzione alla questione è ottenuta grazie all'**invecchiamento**, un campo aggiuntivo che indica per quanto tempo un processo si trova all'interno della ready queue oppure più direttamente un aumento progressivo della priorità in base alla permanenza in coda. Alternativamente sarebbe possibile combinare scheduling con priorità e round-robin per garantire che ogni processo progredisca.

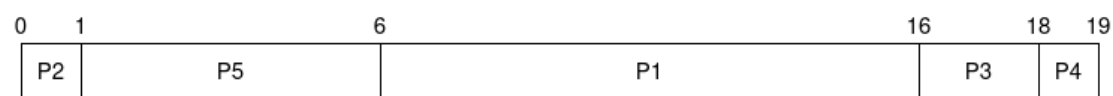


Figura 5.4: Diagramma di Gantt per algoritmo con priorità

Facendo uso del concetto di priorità si potrebbe pensare di implementarlo sulle ready queues. Questa è l'idea di base per l'algoritmo di scheduling a **code multi-livello**.

Si definiscono più code con un valore di priorità statico, nelle quali saranno inseriti i processi scelti dallo scheduler. Questo approccio è spesso usato per suddividere i processi in primo piano da quelli in background, assegnando un valore di priorità maggiore a quelli

interattivi rispetto ai batch. Data inoltre la suddivisione in più entità, consente di definire un algoritmo di scheduling potenzialmente diverso per ogni coda.

Non consentire ai processi di spostarsi fra le code è un approccio rigido, ma ha il vantaggio di avere un basso carico sullo scheduling. In ogni caso, se si volesse dare la possibilità di rompere questo vincolo, staremmo implementando un algoritmo di scheduling a **code multilivello con retroazione**. L'idea consiste nel separare i processi che hanno caratteristiche diverse in termini di CPU burst; se hanno una lunga sequenza di operazioni CPU, saranno spostati in una coda con minor priorità, prediligendo i processi interattivi. Lo spostamento dinamico fra code garantisce un invecchiamento per evitare di incorrere in starvation. Generalmente, questo algoritmo è caratterizzato da alcuni parametri:

- Numero di code
- Algoritmo di scheduling per ciascuna coda
- Metodo per determinare quando spostare un processo in code più prioritarie.
- Metodo per determinare quando spostare un processo in code meno prioritarie.
- Metodo per determinare la coda di priorità nella quale inserire un processo.

Questa dinamica lo rende l'algoritmo più generale di quelli discussi, ma anche il più complesso da implementare.

5.3 Scheduling dei thread

Come già discusso, distinguiamo i thread dei processi tra livello utente e livello kernel; per i sistemi che fanno uso di queste funzioni, non si effettua lo scheduling dei processi, bensì direttamente quello dei thread kernel, mentre i thread utente sono gestiti da una libreria. Per eseguire questi ultimi sarà necessario associarvi un thread kernel, dinamica normalmente ottenuta grazie a un LWP posto fra i due.

Tra più richieste concorrenti di utilizzo dei core della CPU si parla di **ambito della contesa**. In un modello da molti a uno o da molti a molti, la libreria dei thread pianifica l'esecuzione di quelli a livello utente su un LWP libero, **restringendo l'ambito al processo** (PCS), perché la contesa ha luogo fra i thread dello stesso. Alternativamente, il kernel esamina tutti i suoi thread per determinare quale debba essere eseguito, quindi **allargando l'ambito all'intero sistema** (SCS).

Con un PCS, lo scheduling è spesso basato su una priorità assegnata dal programmatore; la libreria generalmente non la modifica, sebbene sia consentito da alcune implementazioni, per poi applicare la prelazione al processo in esecuzione. Per quanto riguarda la libreria Pthreads, è possibile specificare l'ambito della contesa nella fase di

generazione dei thread usando i valori **PTHREAD_SCOPE_PROCESS** per lo scheduling PCS, e **PTHREAD_SCOPE_SYSTEM** per l'SCS. Nei sistemi con modello molti a molti il primo valore pianifica i thread utente sugli LWP disponibili, mentre il secondo crea in corrispondenza di ciascun thread utente un LWP a esso vincolato. Per leggere e impostare gli ambiti, Pthread fornisce le seguenti funzioni:

- `pthread_attr_setscope(pthread_attr_t *attr, int scope)`
- `pthread_attr_getscope(pthread_attr_t *attr, int *scope)`

Dove il primo parametro per entrambe è un puntatore agli attributi del thread, il secondo del setter riceve uno dei due valori di scope mostrati prima per stabilire l'ambito della contesa, mentre il secondo parametro del getter contiene un puntatore a un intero che contiene l'attuale valore dell'ambito della contesa. In caso di errore, ambo le funzioni ritornano valori non nulli. Infine, i sistemi come Linux e macOS consentono di usare soltanto l'ambito esteso al sistema. Segue esempio di creazione thread e assegnazione ambito di contesa con Pthreads.

```

1  # include <pthread.h>
2  # include <stdio.h>
3  # define NUM_THREADS 5
4
5  int main (int argc, char *argv[]) {
6      int scope;
7      pthread_t tid[NUM_THREADS];
8      pthread_attr_t attr;
9
10     // Ottieni gli attributi di default
11     pthread_attr_init(&attr);
12
13     // Definisci l'ambito della contesa
14     if (pthread_attr_getscope(&attr, &scope)) {
15         printf("Impossibile appurare valore ambito contesa");
16     } else {
17         if (scope == PTHREAD_SCOPE_PROCESS) printf("Process");
18         else if (scope == PTHREAD_SCOPE_SYSTEM) printf("System");
19         else printf("Valore ambito contesa non ammesso");
20     }
21
22     // Imposta algoritmo di scheduling PCS o SCS
23     pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
24
25     // Crea e attendi la terminazione dei thread
26     for (int i = 0; i < NUM_THREADS; i++)
27         pthread_create(&tid[i], &attr, runner, NULL);
28     for (int i = 0; i < NUM_THREADS; i++)
29         pthread_join(tid[i], NULL);
30 }
```

```
31 // Funzione eseguita da ogni thread
32 void *runner () {
33     /* Fai qualcosa*/
34     pthread_exit(0);
35 }
```

5.4 Valutazione algoritmi

La scelta dell'algoritmo è ponderata in base a criteri come la massimizzazione dell'utilizzo della CPU e del throughput, e la minimizzazione dei tempi di attesa e di completamento. Una volta definiti, è possibile usufruire di alcune strategie di valutazione, il cui insieme di maggiore utilizzo riguarda la **valutazione analitica**, la quale consente di, dato un input di carico e un algoritmo da applicare, fornire una formula matematica o un valore che valuta le prestazioni ottenute nell'istanza del problema.

La **modellazione deterministica** è il primo metodo di questo insieme; considera un carico predeterminato definito da numeri esatti e definisce le prestazioni dell'algoritmo ritornandone altrettanti. È chiaro che non è sempre possibile avere dati discreti, ragion per cui, per quanto preciso, il metodo soffre limiti di applicazione. Ciò vale non solo per i numeri, bensì anche per i processi; non esiste un loro insieme statico e molti sistemi devono di conseguenza poterne gestire sempre di nuovi. L'idea fornita per la valutazione di algoritmi in questi contesti è quella dell'**analisi delle reti di code**: si richiede il calcolo delle distribuzioni di sequenze di operazioni CPU, I/O e degli istanti di arrivo dei processi in coda per fornire una probabilità che si verifichi una determinata sequenza. Questa è di norma definita dalla sua media esponenziale, come anche la produttività media e altre statistiche utili.

Il sistema di calcolo è descritto come una rete di server, ognuno dei quali con una propria coda d'attesa, mentre la CPU mantiene una ready queue, il sistema I/O e le code di attesa dei dispositivi. In particolare, quando si è in condizioni di equilibrio, il numero medio dei processi presenti nella coda n è uguale al prodotto tra il tasso medio di arrivo λ e il tempo medio di attesa W . Questa formulazione è nota come **Formula di Little**:

$$n = W \times \lambda$$

La formula è molto utile perché funziona per ogni algoritmo di scheduling e distribuzione degli arrivi, rendendo di conseguenza particolarmente funzionale anche l'analisi delle reti di code per il confronto tra algoritmi di scheduling. Tuttavia, capita spesso di avere delle distribuzioni complicate e difficilmente trattabili matematicamente, causando varie approssimazioni che non sempre rispecchiano correttamente le prestazioni reali.

Per una valutazione più precisa si può ricorrere alle **simulazioni**; una tecnica che richiede un sistema di calcolo ad hoc le cui strutture dati rappresentano gli elementi del sistema e, tramite una variabile di clock, consente di osservare l'evoluzione del software

per eventualmente raccoglierne statistiche con un **trace tape**, una registrazione della sequenza di eventi reali del sistema usata come input per la simulazione. Per quanto efficace, è allo stesso tempo una strategia onerosa da attuare, poiché richiede tempo e costi per la codifica, testing e messa a punto del simulatore; risorse che potrebbero essere dedicate al metodo più accurato di tutti: la **realizzazione** all'interno del sistema operativo e la verifica delle sue prestazioni nelle reali condizioni d'uso. Sebbene sia il workflow più realistico, allo stesso tempo presenta alcuni problemi degni di nota, come la sua poca flessibilità; qualsiasi modifica all'algoritmo richiede infatti un intervento diretto sul sistema operativo, oppure la gestione di più programmi in ambienti diversi. Per valutare le prestazioni in quest'ultimo caso in particolare, si esegue uno script per fare eseguire le stesse azioni su diverse piattaforme. Ne consegue che gli algoritmi di scheduling più efficaci in contesti reali siano quelli progettati per adattarsi dinamicamente alle condizioni operative del sistema.

Capitolo 6

Sincronizzazione dei processi

6.1 Sezione critica e Peterson

Come già menzionato, l'esecuzione di processi concorrentemente o in parallelo su dati condivisi può portare a problemi riguardanti l'integrità di questi ultimi; la soluzione proposta nei capitoli precedenti implementava un buffer di dimensione fissa, ma sarebbe possibile anche utilizzare un contatore che si incrementa ad ogni nuovo elemento inserito nel buffer e decrementa quando ne vengono rimossi.

Tuttavia, se il counter è inserito in più file ed è condiviso, è possibile che si presentino comportamenti indesiderati, come per esempio l'aumento e diminuzione della variabile allo stesso tempo, riportando valori errati. Si ritiene dunque necessario fornire una garanzia che un solo processo alla volta possa modificare i dati condivisi, evitando che questi dipendano dall'ordine degli accessi, quindi dalle race condition. La forma presa da queste soluzioni ha il nome di **sincronizzazione e coordinamento tra processi**.

Supponiamo un sistema composto da un totale di n processi, ognuno dei quali con un proprio segmento di codice. Definiamo **sezione critica** la parte del segmento in cui i processi possono accedere e modificare dati condivisi; la soluzione relativa alla gestione di questo spazio consiste nel progettare un meccanismo per regolarne gli accessi. Esistono più sezioni di codice:

- **Sezione di ingresso:** Implementa il meccanismo di sincronizzazione.
- **Sezione di uscita:** Segue la sezione di ingresso.
- **Sezione non critica:** La parte restante del codice.

Il protocollo ideato per il software dovrà inoltre rispettare i seguenti tre requisiti:

1. **Mutua esclusione:** Quando un processo è in esecuzione all'interno della sua sezione critica, nessun altro processo può accedere alla propria.

2. **Progresso:** Se nessun processo è in esecuzione nella propria sezione critica e qualcuno di loro desidera accedervi, solo i processi al di fuori delle relative sezioni critiche possono contribuire alla scelta di quello che potrà accedervi per primo. Questa non è una scelta infinitamente rimandabile.
3. **Attesa limitata:** Se un processo ha fatto richiesta di entrare nella sua sezione critica, si pone un limite al numero di volte in cui gli altri processi possono accedere alle proprie prima che si accordi la richiesta del primo. In tal modo, tutti i processi potranno eventualmente accedere alla sezione.

La soluzione del problema in ambiente single-core è banalmente ottenuta impedendo il verificarsi di interruzioni durante la modifica di dati condivisi, quindi fornendo algoritmi senza prelazione. Tuttavia ciò non si può dire per sistemi multi-core, perché disabilitare gli interrupt potrebbe richiedere molto tempo, impattando anche l'efficienza del sistema.

Le due soluzioni proposte per il problema della sezione critica in ambiente multi-core sono:

- **Kernel con diritto di prelazione:** Consente che un processo in modalità di sistema sia sottoposto a prelazione, rinviandone quindi l'esecuzione.
- **Kernel senza diritto di prelazione:** Non consente di applicare prelazione a processi di sistema attivi, seguitandone di conseguenza l'esecuzione fino a quando non cambia modalità, si blocca, oppure ceda il controllo della CPU volontariamente. Ciò rende questi kernel immuni a race conditions.

Si potrebbe pensare quindi che i kernel senza diritto di prelazione siano i migliori per la risoluzione del problema. Tuttavia, si tende a preferire gli altri data la loro velocità nelle risposte e un miglior adattamento alla programmazione in tempo reale, permettendo di effettuare la prelazione di un processo attivo nel kernel.

Una classica soluzione al problema della sezione critica è l'algoritmo di **Peterson**. Questo è limitato a due processi P_i e P_j , ognuno dei quali esegue alternativamente la propria sezione critica e non critica. È richiesto che condividano una variabile intera *turn*, che indica di chi sia il turno di accesso alla sezione, e un array di booleani *flag*[2], il quale segnala i processi pronti a entrarvi.

Prendendo come esempio il seguente codice del processo P_i , si può vedere come segnali la richiesta di entrare ponendo il proprio flag a vero e dia il turno all'altro processo, per evitare di entrare nella sezione critica quando quest'ultimo la sta utilizzando. Dall'altra parte, al termine dell'esecuzione in sezione critica del processo P_j , porrà il suo flag a falso, consentendo a P_i di entrare; così via fino al termine del software. Inoltre, in caso di richieste contemporanee verrà mantenuta solo l'ultima richiesta.

```

36 // Il seguente codice vale per il processo P-i
37 while (true) {

```

```
38     flag[i] = true;
39     turn = j;
40     while (flag[j] && turn == j);
41
42     // Sezione critica
43
44     flag[i] = false;
45
46     // Sezione non critica
47 }
```

Dimostriamo adesso le tre proprietà che il protocollo deve rispettare per fornire una soluzione al problema della sezione critica: per quanto riguarda la **mutua esclusione**, notiamo come nel codice ogni processo P_i accede alla propria sezione critica se e solo se il flag di P_j è falso oppure il turn ha valore i ; inoltre, se entrambi i processi sono eseguibili in concomitanza nelle rispettive sezioni critiche, allora entrambi i flag saranno posti a vero. Per queste ragioni è impossibile che P_i e P_j eseguano le proprie istruzioni del ciclo while allo stesso tempo. La variabile *turn*, infatti, non può avere più di un valore e fino al termine della permanenza di P_j nella sua sezione critica rimangono il flag di j a vero e turn con il valore di j , confermando la mutua esclusione.

Per il **progresso** e l'**attesa limitata** invece notiamo come l'ingresso di P_i nella propria sezione critica è impedito solo se è bloccato nella sua iterazione while, con variabili flag di j a vero e turn con valore j . Non esistono altre possibilità perché:

- Se P_j non fosse pronto ad entrare in sezione, il flag di j sarebbe falso, permettendo l'accesso a P_i .
- Se il flag di j è vero e il processo sta eseguendo la sua sezione critica, può avere turn con valore i , facendo entrare P_i in critica, oppure turn con valore j , che vede P_j continuare ad eseguirla.

All'uscita dalla sezione, P_j imposterà il suo flag a falso per permettere a P_i di entrare, mentre se P_j lo reimposta a true, dovrà anche porre turn a valore i . Siccome P_i non modifica questi dati mentre è al di fuori della sezione critica, entrerà al suo interno dopo che P_j avrà effettuato più di un ingresso, garantendo gli altri due requisiti.

La soluzione di Peterson posta in questi termini potrebbe tuttavia non funzionare sempre a causa di come i compilatori e i processori lavorano sulle architetture moderne, principalmente perché attuano il riordinamento delle istruzioni di lettura e scrittura per ottimizzare le prestazioni. L'unico modo per preservare la mutua esclusione è l'implementazione di strumenti di sincronizzazione adeguati, trattati nelle sezioni successive.

6.2 Supporto hardware

Con Peterson abbiamo definito una soluzione software per il problema della sezione critica, perché l'algoritmo garantisce mutua esclusione senza dover dipendere dall'hardware. Tuttavia, questa indipendenza è ciò che di base non garantisce il funzionamento su architetture moderne, ragion per cui ci affideremo a meccanismi di sincronizzazione.

Gli elaboratori utilizzano un **modello di memoria**, la modalità per determinare le garanzie sulla memoria da fornire agli applicativi. Ne esistono di due tipi:

- **Fortemente ordinato**: Una modifica alla memoria su un processore è immediatamente visibile anche a tutti gli altri.
- **Debolmente ordinato**: Una modifica alla memoria su un processore non sempre è immediatamente visibile anche a tutti gli altri.

In quanto i modelli variano in base al processore, è necessario che forniscano istruzioni ad hoc per consentire la loro implementazione; tali istruzioni sono dette **barriere di memoria** e alla loro esecuzione, il sistema garantisce che siano eseguite tutte le operazioni di load e store antecedenti alle successive, risolvendo anche il problema del riordinamento delle istruzioni.

Le barriere di memoria fungono quindi da "blocco" per assicurarsi l'esecuzione di un insieme di operazioni prima di procedere con le successive parti del codice; sono operazioni di basso livello spesso utilizzate nei sistemi operativi o dagli sviluppatori dei kernel.

Le moderne architetture forniscono anche un particolare insieme di istruzioni dedito alla manipolazione delle parole di memoria in modo atomico:

- **test_and_set()**: Controlla e modifica il contenuto di una parola di memoria. Imposta atomicamente una variabile a true e restituisce il suo valore precedente, consentendo di rilevare se il lock era già acquisito.
- **compare_and_swap()**: Scambia il contenuto di due parole di memoria. Usa tre operandi: **int *value**, indicante il valore della parola, **int expected**, il valore che ci si aspetta abbia la parola, e **int new_value**, il nuovo valore di quest'ultima, preso dall'altra parola. Se il valore della parola è uguale all'atteso, viene aggiornato con quello nuovo, altrimenti non è effettuata alcuna modifica. In ogni caso, ritorna il value discusso.

Queste due istruzioni sono eseguite atomicamente, quindi in presenza di due loro istanze esecutive su due unità di elaborazione, saranno eseguite in modo sequenziale con un ordine arbitrario. Con questa dinamica e l'aiuto di una variabile booleana globale **lock**, è possibile garantire la mutua esclusione. In particolare, si implementano il lock e un array di booleani globale inizializzati a false; quando un processo richiede l'accesso alla

propria sezione critica, setta la sua cella di array a true e dichiara una variabile key a valore 1. Fintanto che presentano questa configurazione, ovvero fino a quando la sezione critica è utilizzata da un altro processo, il flusso rimane in un ciclo while fino a quando key non è aggiornata con la compare and swap. Terminata l'iterazione, la cella dell'array sarà posta a false e il processo entrerà nella sua sezione critica.

Per quanto riguarda l'attesa limitata, invece, si itera l'array in ordine ciclico e setta il primo processo in attesa come il prossimo che può entrare nella propria sezione critica; quindi qualunque processo voglia accedervi, potrà farlo entro $n - 1$ turni.

```

1  while (true) {
2      waiting[i] = true;
3      key = 1;
4
5      while (waiting[i] && key == 1) {
6          key = compare_and_swap(&lock, 0, 1);
7      }
8      waiting[i] = false;
9
10     // Sezione critica
11
12     j = (i+1) % waiting_len;
13     while ((j != i) && !waiting[j]) j = (j+1) % waiting_len;
14
15     if (j == i) lock = 0;
16     else waiting[j] = false;
17
18     // Sezione non critica
19 }
```

In genere, dato che sono operazioni atomiche, test&set e compare&swap sono utilizzate come base per costruire strumenti di sincronizzazione più astratti, come le **variabili atomiche**, le quali forniscono operazioni su tipi di dati base, come interi e booleani. Possono essere utilizzate per garantire mutua esclusione in alcune situazioni dove possono presentarsi race condition. Sono usate comunemente nei sistemi operativi e applicazioni concorrenti.

6.3 Lock Mutex, Semafori e Monitor

Le soluzioni hardware possono essere astratte in strumenti software più accessibili come il **Lock Mutex**, usato per proteggere le sezioni critiche e prevenire le race condition.

La procedura associata vede un processo acquisire il lock prima di entrare nella regione, e rilasciarlo quando ne esce. Il meccanismo è ottenuto grazie alle funzioni **acquire()**, che acquisisce il lock quando è disponibile e pone i processi che lo richiedono in attesa attiva quando non lo è, fin quando non viene rilasciato con **release()**. Queste devono

essere eseguite atomicamente e lo stato del lucchetto è consultabile grazie a una sua variabile booleana **available**.

```

1 // Definizione di acquire
2 acquire () {
3     while (!available) { /* Attendi */
4         available = false;
5     }
6
7 // Definizione di release
8 release () {
9     available = true;
10 }

```

Il vantaggio di questo meccanismo è di non rendere necessario alcun cambio di contesto, che tendenzialmente occupa molto tempo, tuttavia, se molti processi richiedono di acquisire un lock indisponibile, questi continueranno a ciclare e chiamare `acquire()` fino al suo rilascio, sprecaendo cicli della CPU. Per questa ragione sono chiamati anche "spinlock" e vengono utilizzati in sistemi multiprocessore dove possono far girare i thread su core diversi mentre il detentore del lock esegue la sua sezione critica.

Uno strumento più robusto per gestire la sincronizzazione dei processi è quello dei **semafori**; sono delle variabili intere a cui si può accedere¹ solo tramite due operazioni atomiche chiamate `wait()` e `signal()`, definite come segue:

```

1 wait (S) {
2     while (S <= 0) {
3         // Attesa attiva
4     }
5     S--;
6 }
7
8 signal (S) { S++; }

```

Tutte le modifiche al valore del semaforo S devono essere effettuate all'interno di queste due funzioni e in modo indivisibile. Inoltre, nel caso di una chiamata a `wait(S)`, sarà necessario verificare anche il valore intero del semaforo e la sua eventuale modifica. Distinguiamo due tipi di semafori:

- **Binari:** Il loro valore è limitato a $[0, 1]$; in quanto simili ai lock mutex, vengono usati al loro posto dove i primi non possono essere applicati.
- **Contatore:** Il loro valore è un numero intero. Sono utilizzati in un contesto dove è necessario un controllo dell'accesso ad una risorsa presente in più esemplari.

In particolare, per questi ultimi, si inizializzano con il valore delle risorse disponibili, e i processi che ne desiderano utilizzare una dovranno invocare `wait(S)`, decrementandone

¹Escludendo l'inizializzazione.

quindi il valore, mentre coloro che devono restituirne, useranno *signal(S)*, aumentandolo. Nel caso in cui il semaforo valga 0 significa che tutte le risorse sono occupate e i processi richiedenti dovranno **bloccarsi** fin quando il semaforo non torna positivo.

Ciò introduce nuovamente il problema dell'attesa attiva, quindi si va a modificare le definizioni delle due operazioni elementari. Per quanto riguarda *wait()* possiamo bloccare il processo e porlo nello stato di attesa, sicché lo scheduler possa fare un cambio di contesto e dare il controllo della CPU a un altro processo pronto. Questo processo sospeso, in attesa del semaforo, sarà riavviato in seguito ad una chiamata a *signal(S)* da parte di un altro processo e tramite un'operazione chiamata *wakeup()* si modifica il suo stato da attesa a pronto, spostandolo nella relativa coda. In un contesto tale, ridefiniamo il semaforo come segue:

```

1  typedef struct {
2      int value; // Valore del semaforo
3      struct process *list; // Processi in attesa
4  } semaphore;

```

Mentre in pseudocodice, *wait* e *signal* diventano:

```

1  wait (semaphore *S) {
2      S->value--;
3      if (S->value < 0) {
4          // Aggiungi il processo a S->list
5          sleep();
6      }
7  }
8
9  signal (semaphore *S) {
10     S->value++;
11     if (S->value <= 0) {
12         // Rimuovi un processo da S->list
13         wakeup();
14     }
15 }

```

La funzione *sleep()* sospende il processo che la invoca, mentre *wakeup()* lo pone come pronto; queste sono due operazioni fornite dal sistema operativo tramite system call.

Contrariamente alla precedente definizione di semaforo, che lo mostrava come un intero, con queste modifiche è possibile scendere a numeri negativi, il cui valore assoluto rappresenta il numero di processi in attesa che il semaforo si liberi. La lista dei processi, invece, è tipicamente implementata con un puntatore a una coda FIFO contenente ciascun PCB.

Consideriamo ora le operazioni di *wait()* e *signal()*; in quanto è richiesto che siano atomiche, si deve garantire che nessun processo le attui allo stesso tempo su uno stesso semaforo; ciò è riconducibile a un problema di accesso alla sezione critica. Nei sistemi a singola CPU banalmente si inibiscono gli interrupt durante le esecuzioni delle due operazioni, ma nei sistemi multi-core disabilitare le interruzioni porterebbe a un drastico

calo delle prestazioni; per questa ragione mettono a disposizione diverse tecniche di realizzazione dei lock, come spinlock o *compare_and_swap()*.

Notiamo infine che la nuova definizione di *wait()* e *signal()* non elimina completamente il problema dell'attesa attiva, bensì lo rimuove dalle sezioni di ingresso degli applicativi; poiché infatti questa si limita alle sezioni critiche delle due operazioni, queste sono in genere convenientemente codificate, sicché siano particolarmente brevi. Ciò fa presentare raramente l'attesa attiva e quando accade, lo fa per breve tempo.

Nonostante i semafori siano un costrutto pratico ed efficace, un loro uso scorretto può causare vari problemi di difficile individuazione, in quanto si possono manifestare solo in presenza di particolari sequenze di esecuzione. Non escludono nemmeno il verificarsi di problemi di sincronizzazione; supponiamo infatti il problema della sezione critica dove tutti i processi condividono una variabile semaforo mutex inizializzata a 1. Ogni processo chiama *wait(mutex)* all'entrata nella propria sezione critica e invoca *signal(mutex)* quando ne esce. Se la sequenza non è rispettata possono verificarsi i seguenti problemi:

- **Ordine della operazioni inverso:** Più processi possono eseguire le proprie sezioni critiche, violando la mutua esclusione.
- **Sostituzione di *signal(mutex)* con *wait(mutex)*:** Il processo si bloccherà permanentemente alla seconda chiamata *wait()*, perché il semaforo non sarà più disponibile.
- **Omissione di una o entrambe le operazioni:** Si viola la mutua esclusione oppure il processo si blocca indefinitamente.

Una strategia per la gestione di questi errori consiste nell'incorporare ulteriori strumenti di sincronizzazione astratti, come il tipo **monitor**.

Un monitor è un **tipo di dato astratto** (ADT) che incapsula i dati mettendo a disposizione un insieme di funzioni, contraddistinte dalla mutua esclusione, per operare su di essi. Contiene inoltre la dichiarazione delle variabili i cui valori definiscono lo stato dell'istanza del monitor. Generalmente, la sintassi di un monitor in pseudocodice è:

```
1  monitor monitorName {  
2      // Dichiarazione di variabili condivise  
3  
4      function P1 (...) { ... }  
5      function P2 (...) { ... }  
6  
7      // Altre funzioni  
8  
9      function Pn (...) { ... }  
10     initCode (...) { ... }  
11 }
```

Le funzioni dentro al monitor hanno accesso solo alle variabili locali o ai parametri formali. Il costrutto assicura inoltre che al suo interno possa essere attivo un solo processo alla volta grazie ad eventuali variabili **condition** x , costrutti sui quali sono applicabili esclusivamente le operazioni di *wait()* e *signal()*.

Entrambe mantengono la stessa funzione discussa con i semafori, ma la seconda non ha effetto in assenza di processi sospesi. Supponiamo che un processo P chiami $x.signal()$ e che ne esista un altro Q associato alla condition x . Se si permette a Q di riprendere la propria esecuzione, l'altro processo P dovrà necessariamente attendere. Tuttavia, concettualmente, potrebbero entrambi continuare l'esecuzione, creando due possibilità:

- **Segnalare e attendere:** P attende che Q lasci il monitor o attenda su un'altra condition.
- **Segnalare e continuare:** Q attende che P lasci il monitor o attenda su un'altra condition.

Non esiste un metodo migliore rispetto all'altro, ma per le implementazioni useremo la prima istanza. Presentiamo una possibile realizzazione di un monitor tramite semafori. A ogni monitor sarà associato un semaforo mutex inizializzato a 1 per garantire la mutua esclusione. Un processo eseguirà *wait(mutex)* prima di entrare nel meccanismo e chiamerà *signal(mutex)* quando uscirà dal monitor. In quest'ultimo scenario, dato che il processo chiamante dovrà attendere finché quello risvegliato si metta in attesa o lasci il monitor, si introduce un nuovo semaforo *next* inizializzato a 0, su cui i processi segnalanti possono autosospendersi. Il relativo contatore si implementa con una variabile intera *next_count*. Si richiede quindi che ogni procedura esterna F sia compresa nel seguente codice:

```

1  wait(mutex);
2
3  // Corpo di F
4
5  if (next_count > 0) signal(next);
6  else signal(mutex);

```

Ciò assicura la mutua esclusione dentro al monitor. La realizzazione delle condition vede poi l'introduzione di un semaforo x_sem e un intero x_count , ambo inizializzati a 0. Quindi, le operazioni di *wait()* e *signal()* associate alla condition x si ridefiniscono come segue:

```

1  wait () {
2      x_count++;
3      if (next_count > 0) signal(next);
4      else signal(mutex);
5      wait(x_sem); // Chiamata a wait di semaforo, non condition
6      x_count--;
7  }
8

```

```

9      signal () {
10         if (x_count > 0) {
11             next count++;
12             signal(x_sem); // Chiamata a signal di semaforo, non
condition
13             wait(next); // Idem qua
14             next_count--;
15         }
16     }

```

È necessario ora decidere l'ordine di ripresa dei processi sospesi alla condition x . Generalmente si crea un costrutto di **attesa condizionale** con forma $x.wait(c)$, dove c è un valore di **priorità** determinato da un'espressione valutata al momento dell'esecuzione di $wait()$. Tale numero è poi associato a un processo sospeso, e all'esecuzione di $x.signal()$ si riprenderà dal processo con la priorità più alta in base alla convenzione scelta.

Di base, tuttavia, il monitor non garantisce il rispetto di questo ordine, poiché potrebbero accadere i seguenti scenari:

- Un processo accede alla risorsa senza prima ottenere il permesso.
- Ottenuto l'accesso alla risorsa, un processo potrebbe non rilasciarla più.
- Un processo tenta di rilasciare una risorsa che non ha mai richiesto.
- Un processo può richiedere più volte la stessa risorsa senza averla rilasciata prima delle successive richieste.

È evidente che bisogna concentrarsi su eventuali soluzioni in merito alla gestione delle operazioni ad alto livello, controllando i programmi che utilizzano il monitor in tal modo che si verifichino due condizioni: in primo luogo, i **processi utenti** devono sempre impiegare il costrutto seguendo la **sequenza corretta**; in secundis, bisognerà assicurare che eventuali **processi non cooperativi** non aggirino la **mutua esclusione** offerta.

Rispettando le due condizioni si ha la certezza di non avere problemi di sincronizzazione, ma è applicabile agevolmente solo per sistemi di piccole dimensioni.

6.4 Liveness

L'impiego di strumenti di sincronizzazione può avere conseguenze relative all'attesa indefinita a determinate risorse. Con **liveness** intendiamo un insieme di proprietà che un sistema deve soddisfare per evitare questo problema e garantire che ogni processo progredisca la sua esecuzione durante il proprio ciclo vitale. Strumenti come i lock mutex e semafori consentono di indurre i problemi di **mancaanza di liveness** alla programmazione concorrente. Analizziamo ora alcune istanze di questo tipo di problemi con relative soluzioni.

Il primo caso di studio è rappresentato dallo **stallo** (deadlock); una situazione nella quale uno o più processi rimangono in eterna attesa del verificarsi di un evento. In particolare, nelle implementazioni dei semafori con coda di attesa, i processi aspetteranno una chiamata a *signal()* impossibile da attuare.

Supponiamo due processi P_0, P_1 , ognuno dei quali con un accesso a due semafori S, Q , ambo impostati al valore 1. Se i due processi chiamano *wait(S)* e *wait(Q)* rispettivamente allo stesso tempo, allora entrambi si bloccheranno fino alla ricezione di *signal()*, ma essendo in attesa, non possono effettuare chiamate a funzioni, risultando in stallo. La soluzione è fornita con un opportuno utilizzo della sincronizzazione.

Un'altra istanza di liveness failure si può ottenere con lo scheduling dei processi. Se è implementato un meccanismo basato sulla priorità, è probabile che un processo a priorità alta richieda dei dati utilizzati da un altro con priorità minore, ed essendo i dati a livello kernel protetti da un lock, il primo processo sarà costretto ad attendere fino al rilascio delle suddette risorse.

La soluzione a questo problema, detto **inversione della priorità**, si ha implementando un protocollo per l'**ereditazione delle priorità**. Fondamentalmente, quando un processo a bassa priorità utilizza dei dati che possono essere richiesti da altri processi più importanti, eredita il livello di priorità di questi ultimi per tutto il tempo in cui usufruiscono di queste risorse, impedendo che processi a priorità intermedia sottraggano la CPU al processo a bassa priorità che detiene il lock, riducendo così il tempo di attesa del processo ad alta priorità.

6.5 Esempi di sincronizzazione

Analizziamo ora alcuni classici problemi in merito al controllo della concorrenza; vengono utilizzati per verificare le proposte di nuovi schemi di sincronizzazione. Il primo, già trattato in precedenza, riguarda il **produttore/consumatore** con memoria limitata, generalmente usato per illustrare le primitive. Data una quantità di memoria limitata definita da un buffer con n posizioni, si condividono le seguenti strutture dati:

```

1  int n; // Numero posizioni buffer
2  semaphore mutex = 1; // Garantisce mutua esclusione accessi buffer
3  semaphore empty = n; // Numero posizioni vuote in buffer
4  semaphore full = 0; // Numero posizioni piene in buffer

```

I prodotti generati saranno inseriti nel buffer e viene segnalato l'utilizzo della posizione, mentre per consumarli si prelevano dal buffer e viene riportato che la posizione è nuovamente libera. I due processi sono quindi simmetrici.

```

1  // Struttura processo produttore
2  while (true) {
3      . . .
4      /* Produci un elemento in next_produced */

```

```

5      . . .
6      wait(empty);
7      wait(mutex);
8      . . .
9      /* Inserisci next_produced in buffer */
10     . . .
11     signal(mutex);
12     signal(full);
13 }
14
15 // Struttura processo consumatore
16 while (true) {
17     wait(full);
18     wait(mutex);
19     . . .
20     /* Rimuovi un elemento da buffer e mettilo in next_consumed */
21     . . .
22     signal(mutex);
23     signal(empty);
24     . . .
25     /* Consuma l'elemento in next_consumed */
26     . . .
27 }

```

Il secondo problema presentato è quello dei **lettori/scrittori**; si supponga una base di dati condivisa da più processi concorrenti, alcuni dei quali richiedenti solo funzioni di lettura (**lettori**), mentre altri anche di scrittura (**scrittori**). Se due lettori accedono contemporaneamente alla propria sezione critica, non si presenta alcun problema; caso contrario se due scrittori devono aggiornare l'area allo stesso tempo.

È necessario impedire quest'ultima istanza garantendo un accesso esclusivo agli scrittori quando questi debbano aggiornare i dati. Questo problema ha molte varianti, ma presentiamo le prime due classiche:

- **Primo problema:** Nessun lettore deve attendere per accedere ai dati, a meno che uno scrittore non abbia ottenuto il permesso di modificare i dati.
- **Secondo problema:** Gli scrittori pronti devono eseguire e terminare l'operazione di scrittura dati condivisi il prima possibile.

In quanto queste due situazioni possono presentare attese indefinite, è necessario modificare la soluzione proposta in base alla situazione. Presentiamo quella relativa al primo problema, che vede condivise le seguenti strutture dati:

```

1     semaphore rw_mutex = 1; // Garantisce mutua esclusione per
    scrittori
2     semaphore mutex = 1; // Garantisce mutua esclusione all'
    aggiornamento di read_count
3     int read_count = 0; // Numero processi che stanno leggendo i dati

```

Se uno scrittore si trova in sezione critica ed n lettori attendono di entrare nella propria, si accoda un lettore a *rw_mutex* e $n - 1$ lettori a *mutex*. Inoltre, quando uno scrittore chiama *signal(rw_mutex)*, lo scheduler potrà scegliere se dare gli accessi ad un altro scrittore oppure ai lettori in attesa.

```

1  // Struttura processo scrittore
2  while (true) {
3      wait(rw_mutex);
4      . . .
5      /* Esegui scrittura */
6      . . .
7      signal(rw_mutex);
8  }
9
10 // Struttura processo lettore
11 while (true) {
12     wait(mutex);
13     read_count++;
14     if (read_count == 1) wait(rw_mutex);
15     signal(mutex);
16     . . .
17     /* Esegui lettura */
18     . . .
19     wait(mutex);
20     read_count--;
21     if (read_count == 0) signal(rw_mutex);
22     signal(mutex);
23 }
```

Le soluzioni a questo problema sono generalizzate in alcuni sistemi mediante **lock di lettura-scrittura**, i quali sono acquisiti grazie ad una specifica della funzionalità richiesta. Si permette a più processi di acquisire più lock in modalità lettura, ma solamente a uno alla volta in modalità scrittura, data la necessità dell'accesso esclusivo.

L'ultimo problema proposto è quello dei **cinque filosofi**: si suppongano cinque filosofi che trascorrono la loro vita a pensare e mangiare. Questi condividono una tavola rotonda al cui centro è posto del riso. La tavola presenta cinque posti disponibili, ma solo un totale di cinque bacchette; una posta a sinistra e l'altra a destra del piatto, entrambe necessarie per mangiare. Se un filosofo pensa, non interagisce, mentre se gli viene fame, cerca di prendere le bacchette più vicine al suo posto. Può prendere una bacchetta libera alla volta e se ne ha due in mano potrà mangiare senza essere interrotto. Terminato il pasto, le lascerà nuovamente disponibili per tornare a pensare.

Questo problema è un classico esempio di gestione della concorrenza dove si richiede la necessità di assegnazione delle risorse ai vari processi evitando stallo e starvation. Una soluzione di questo problema si ottiene grazie ai semafori; i filosofi condivideranno un array *chopstick*[5] dove gli elementi sono inizializzati a 1; si prenderanno le bacchette chiamando *wait()* e le si poseranno con *signal()* sui semafori appropriati. Questa proposta

garantisce che due vicini non mangino allo stesso tempo, ma non esclude eventuali stalli, perché se tutti i filosofi hanno fame allo stesso tempo, alcuni dovranno necessariamente aspettare. Per questa ragione si implementa uno dei seguenti espedienti:

- Solo quattro filosofi possono stare contemporaneamente a tavola.
- Un filosofo può prendere le bacchette ad esso adiacenti solamente se sono entrambe disponibili.
- Soluzione asimmetrica dove i filosofi dispari prendono prima la bacchetta di sinistra e poi quella di destra e viceversa per quelli pari.

```
1 // Struttura dei filosofi
2 while (true) {
3     wait(chopstick[i]);
4     wait(chopstick[(i+1)%5]);
5     . . .
6     /* Mangia */
7     . . .
8     signal(chopstick[i]);
9     signal(chopstick[(i+1)%5]);
10    . . .
11    /* Pensa */
12    . . .
13 }
```

Capitolo 7

Stallo dei processi

7.1 Stallo e multithread

7.2 Caratterizzazioni di stallo

7.3 Metodi di gestione e prevenzione

7.4 Elusione e rilevamento stallo

7.5 Ripristino

Capitolo 8

Bash

8.1 Programmazione Bash

Capitolo 9

API Posix

9.1 Gestione dei file

9.2 Gestione dei diritti

9.3 Lettura/scrittura

9.4 Gestione dei processi

9.5 Segnali

9.6 Comunicazione tra processi tramite PIPE

9.7 Pthread